



清华大学
Tsinghua University

高性能计算导论

第11讲：性能分析与调试

翟季冬
计算机系

目录

- 性能重要性
- 性能分析
 - 典型方法
 - 常用工具
- 正确性调试
 - 典型方法
 - 常用工具
- 总结

性能重要性

性能重要性

- ▶ 高性能计算机是**国之重器**，支撑着关系国计民生的重大科学应用
- ▶ 系统**规模庞大**、**硬件性能高**
- ▶ 性能和功能同等重要

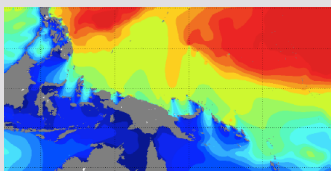
全国人大《“十四五”规划和2035年远景目标纲要》

第三篇：强化算力统筹智能调度，建设若干国家枢纽节点和大数据中心集群，**建设E级和10E级超级计算中心**

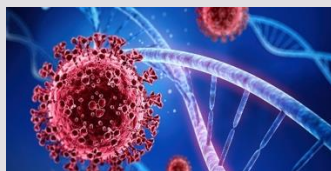
尖端应用



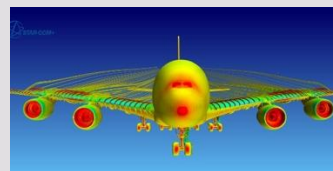
核爆模拟



气候模拟



生物制药



大飞机研制

算力支撑

高性能计算机

美国 Frontier

峰值性能：每秒1.686EFLOPS



中国 神威·太湖之光

峰值性能：每秒125PFLOPS



日本 富岳

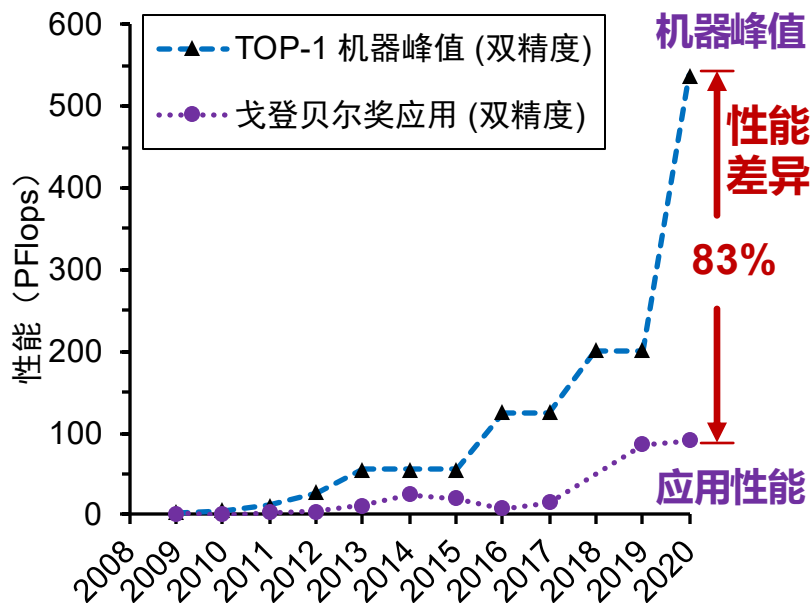
峰值性能：每秒537PFLOPS



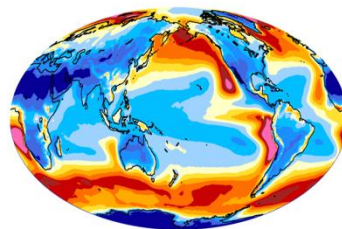
性能意义

尽管高性能计算机硬件性能高，但大量应用程序实际运行性能低下，无法满足解决尖端科学问题的需求

TOP-1 机器峰值 v.s. “戈登贝尔” 奖性能



戈登·贝尔奖：国际高性能方向应用最高奖



大气模式应用



神威·太湖之光

累计投入超10人年才扩展到150万核，
仅达全机规模的15% (全机1000万核)

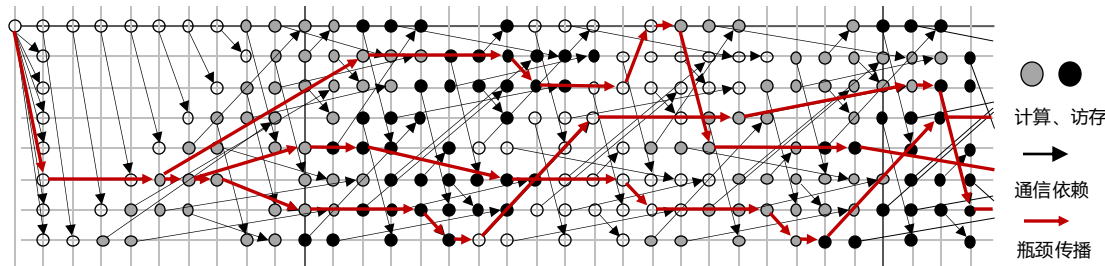
国产高性能计算机上应用并行优化

如何提升大规模应用程序的性能，是高性能计算领域世界难题

面临挑战

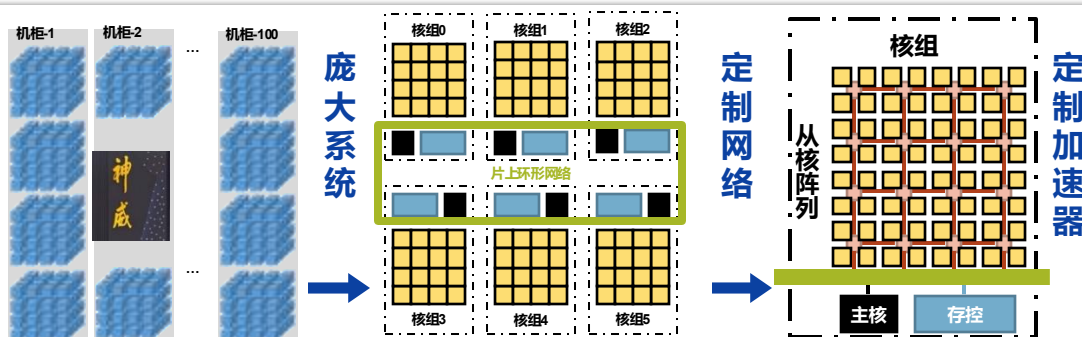
并程序性能难以提升的挑战：瓶颈定位难、程序扩展难、并行优化难

瓶颈定位难



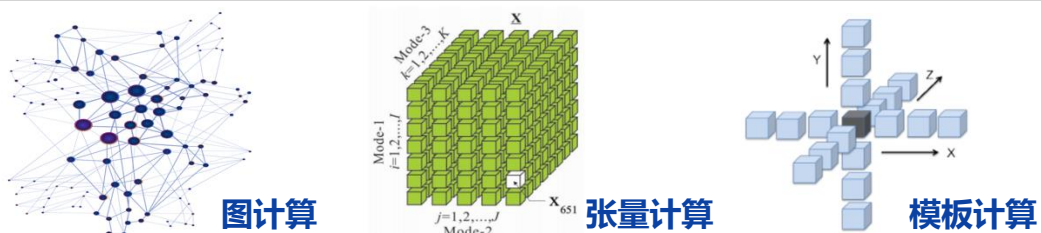
并程序的计算、
访存、通信之间
依赖错综复杂

程序扩展难



系统规模大导致
通信同步开销大、
定制硬件导致并
行算法设计难

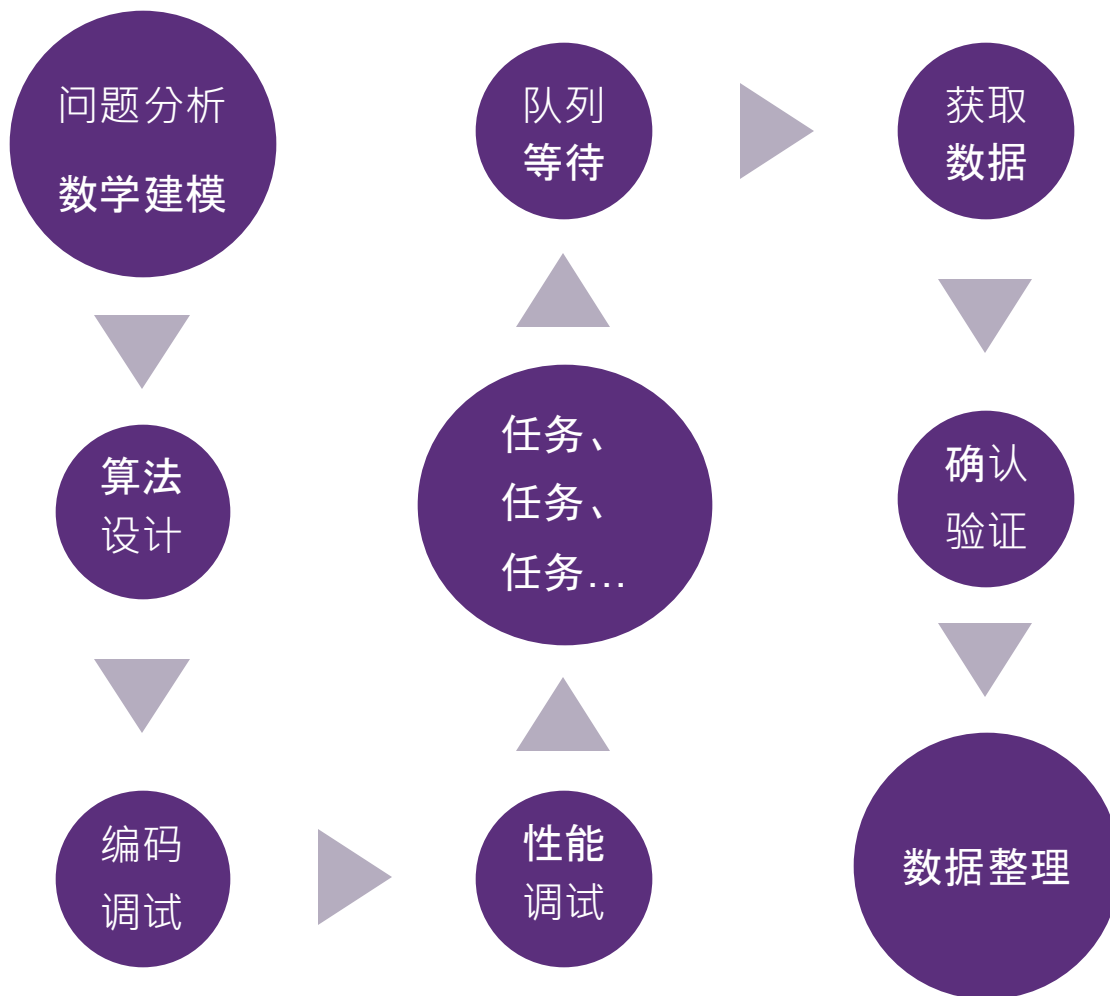
并行优化难



应用程序负载特
征多样、每个应
用都需深入优化

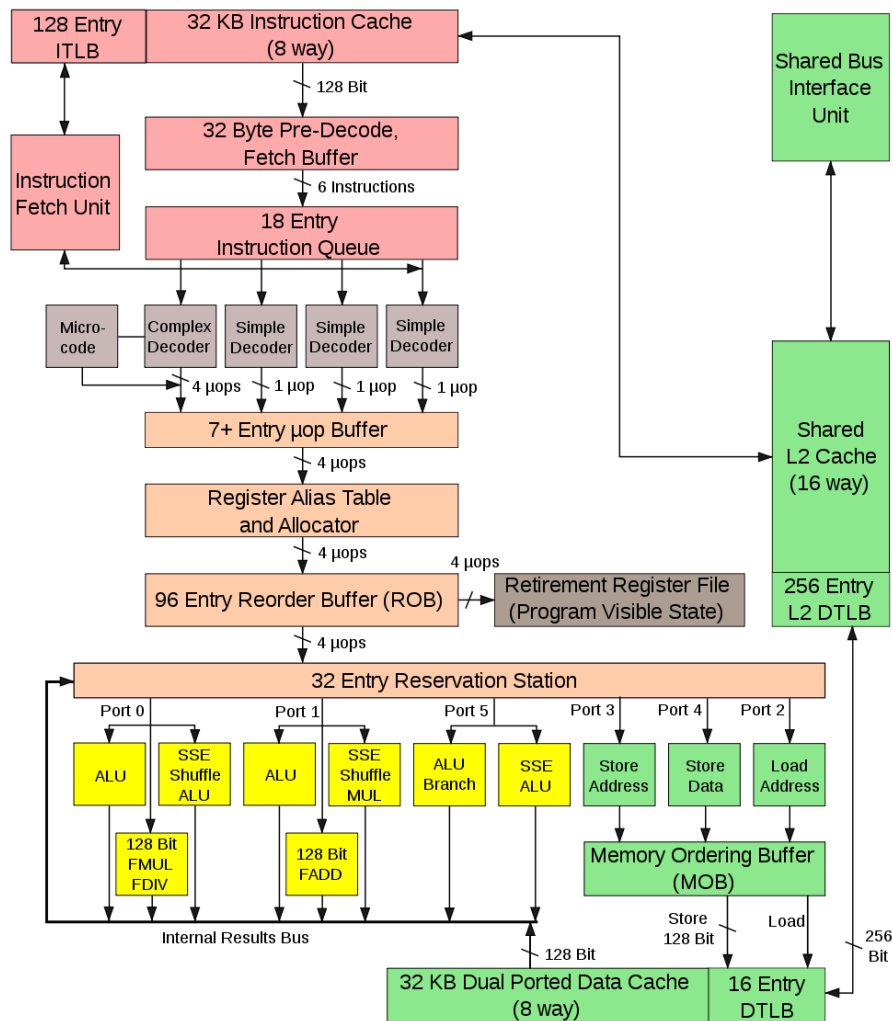
性能是一个系统工程

- 性能涉及系统开发**每一个环节**



性能影响因素 - 计算

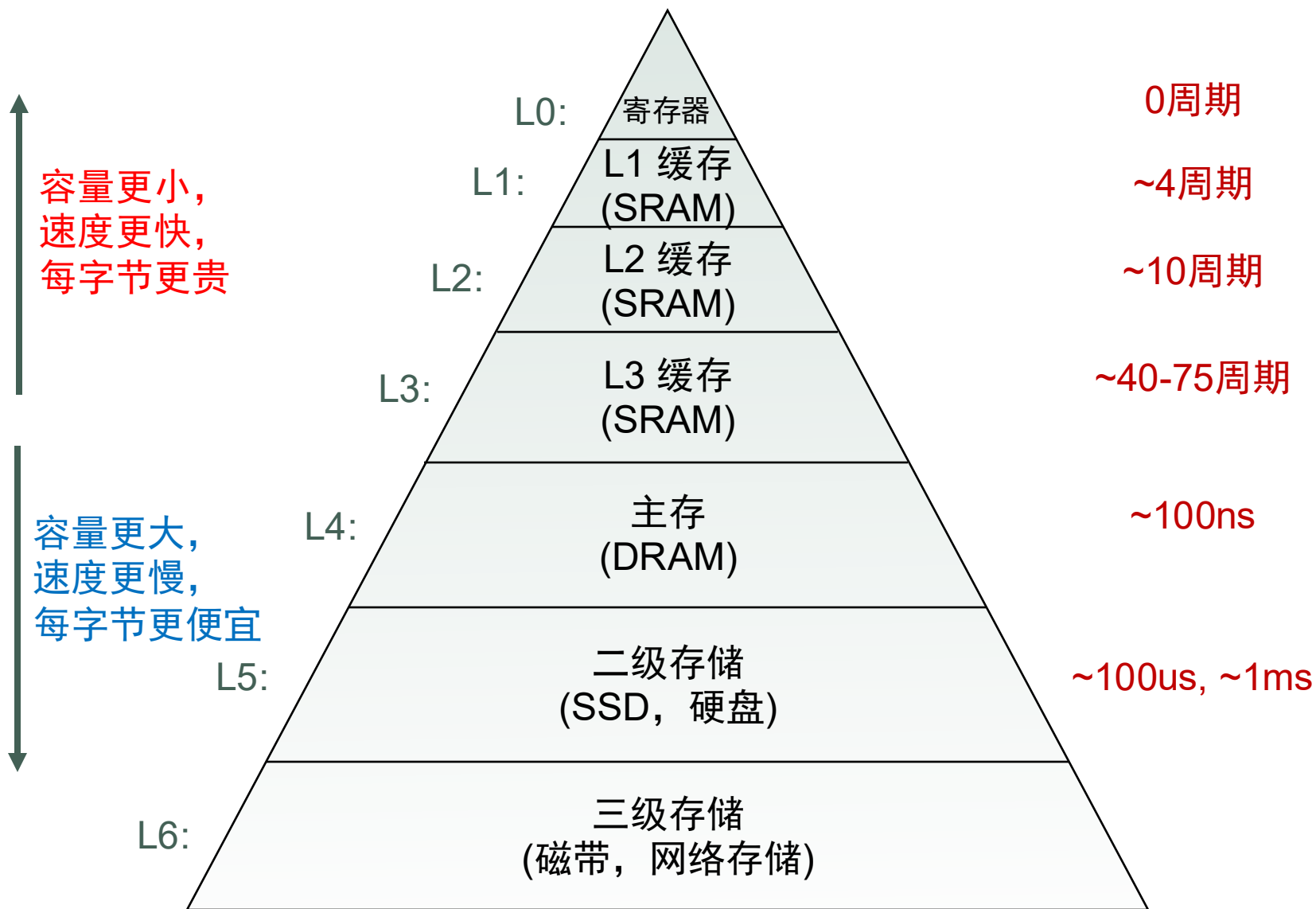
■ 处理器性能因素



Intel Core 2 Architecture

Intel 微体系架构

性能影响因素 - 数据访问



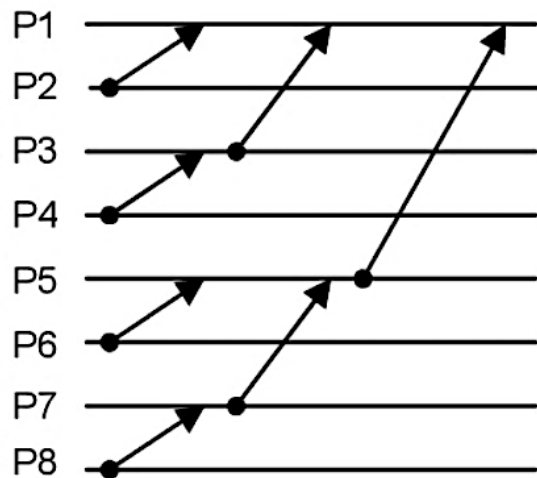
性能影响因素 - 通信性能

■ 通信硬件性能：

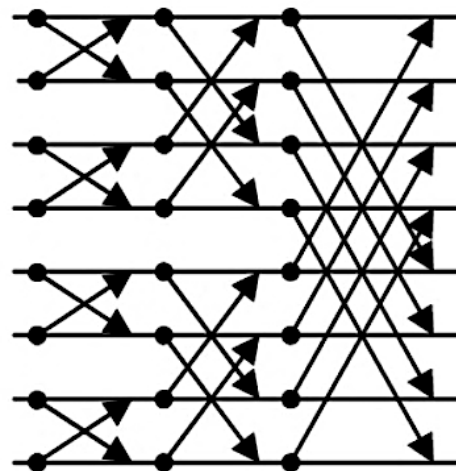
- 网络延迟、带宽、网络拓扑

■ 程序通信模式：

- 计算通信重叠
 - 利用非阻塞通信，可以在通信的同时进行计算任务
- 任务之间同步依赖、负载均衡等



二叉树模式



蝴蝶模式
(Butterfly pattern)

典型性能优化方法

■ 单机部分：

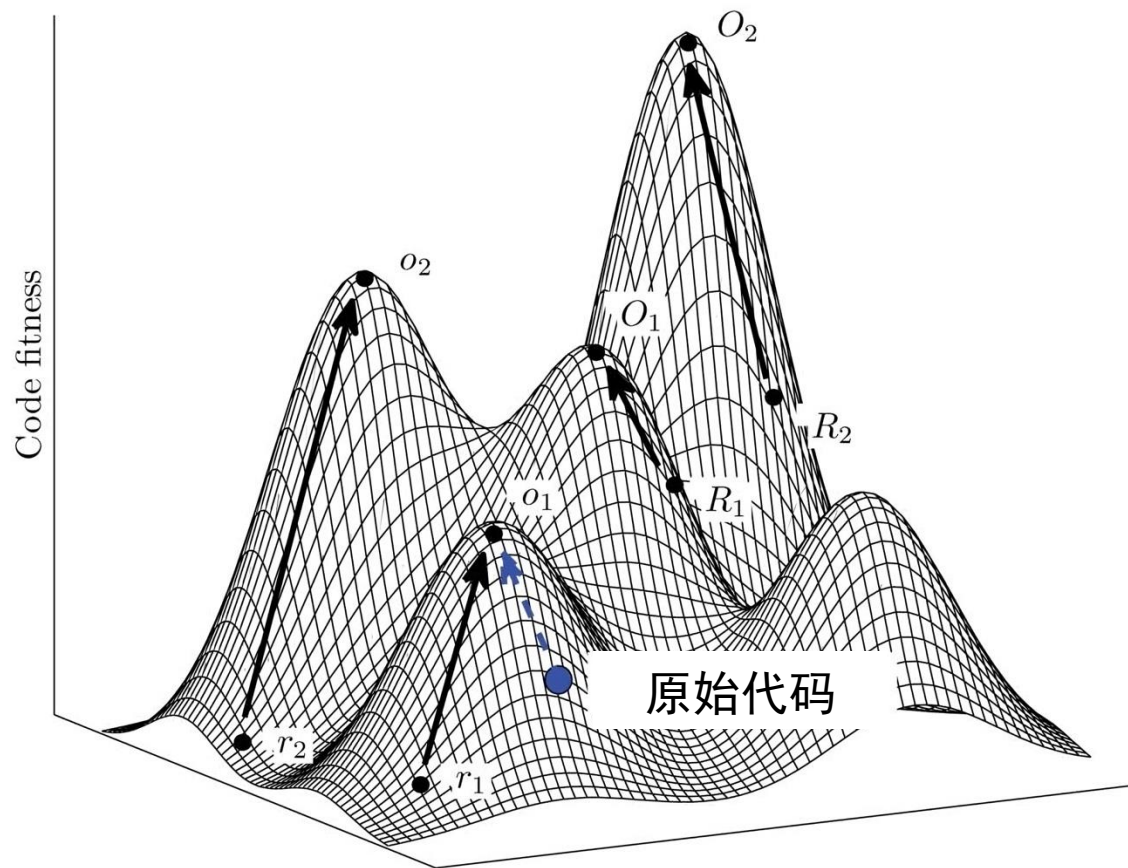
- 探索指令级并行
- 填充流水线
- 利用向量指令 SIMD
- 使用 Cache 数据
- 提高程序局部性
- 大块 I/O 访问

■ 并行部分：

- 探索任务级并行
- 改善负载均衡
- 降低延迟影响（汇总小消息）
- 最大计算通信比
- 平衡 I/O 访问策略

性能优化的过程

- 性能优化是一个不断迭代过程



Performance matters, but be wise



**"Premature optimization is
the root of all evil." [K79]**

Donald E. Knuth、高德纳、74年图灵奖

The Decline of Performance Engineering



**“The First Rule of Program Optimization: Don't do it.
The Second Rule of Program Optimization - For experts only: Don't do it yet.” [J88]**

Michael A. Jackson 英国计算机科学家

性能指标

性能指标 Metric

- 性能指标：
 - 与程序性能相关的统计量
 - 描述程序行为特征
 - 程序员采用性能工具获取相应性能指标
 - 帮助理解性能、优化性能
- 按类型分类：
 - 计算
 - 通信
 - 访存
 - I/O

常用性能指标

■ 计算

- 指令分布（浮点、整数、分支等）
- 指令级并行度
- 分支预测成功率
- 处理器利用率

■ 访存

- Load/Store 数量
- Cache 缺失率
- TLB 缺失率
- 缺页次数
- 工作集的大小

■ 通信

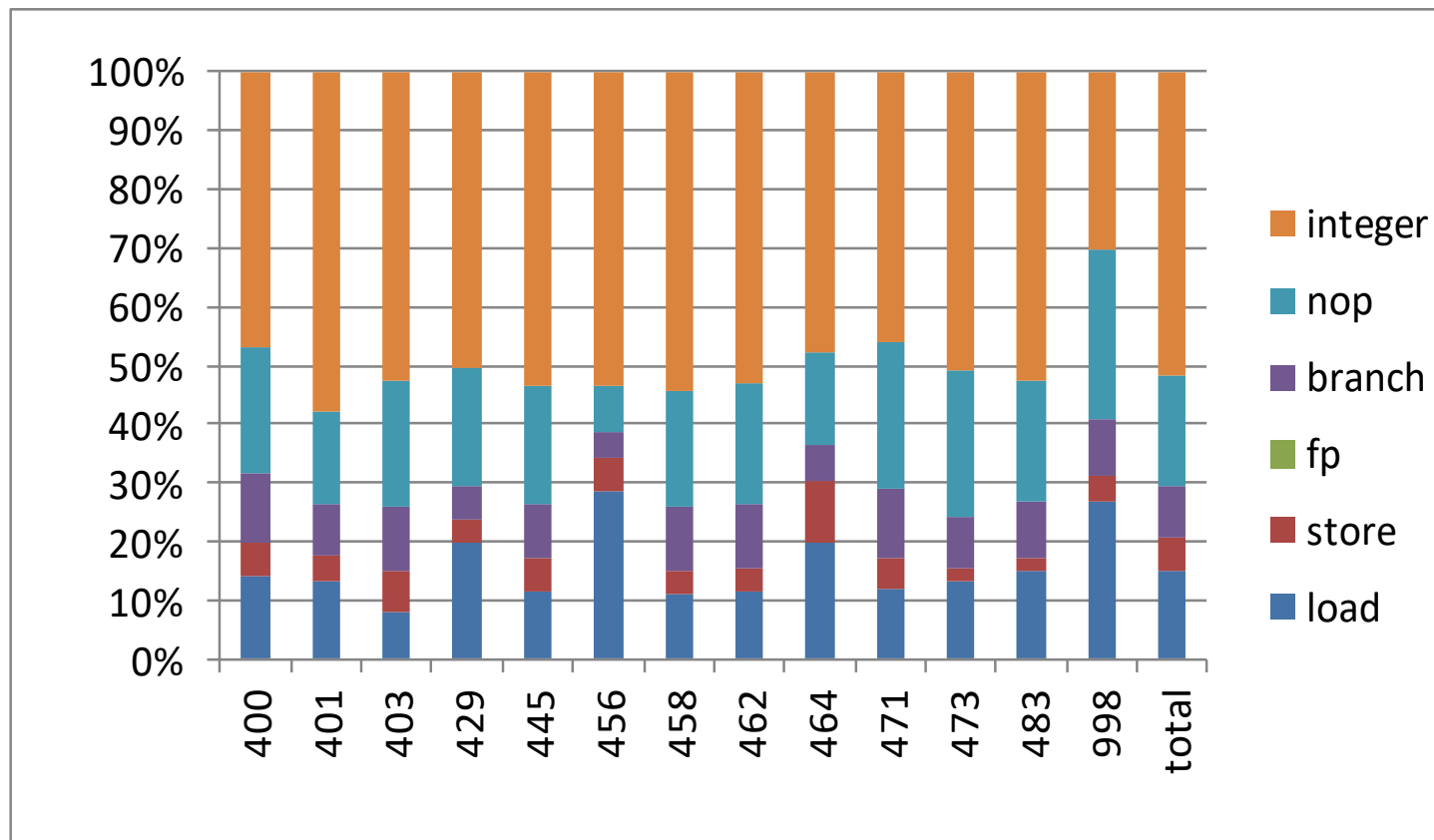
- 通信类型
- 消息大小
- 消息频率
- 网络带宽利用率
- MPI：通信函数用时和调用次数

■ I/O

- I/O 请求类型
- I/O 请求频率
- I/O 请求大小
- 访问模式（顺序、随机访问）
- 磁盘利用率

SPEC CPU指令特征

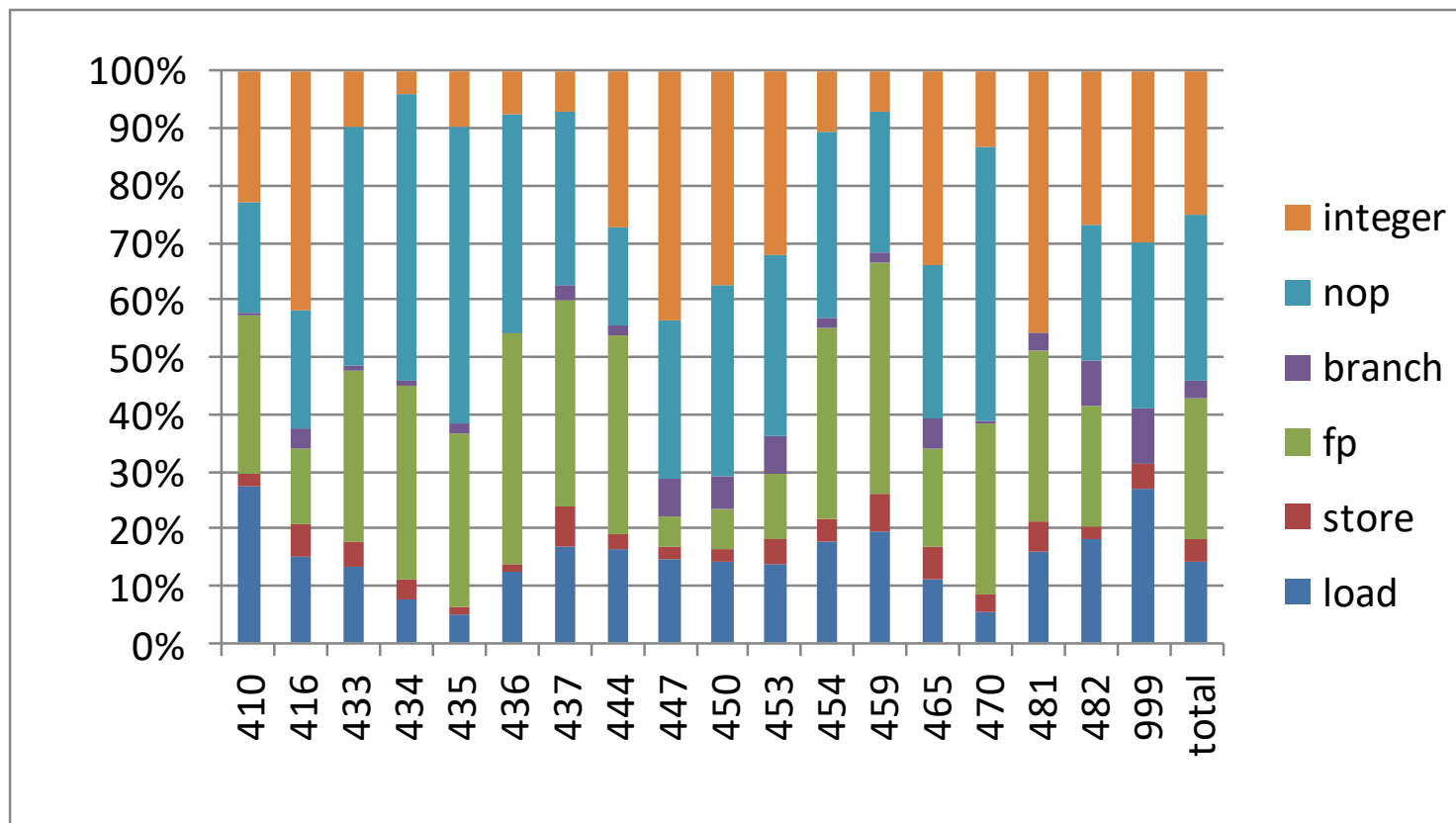
SPEC CPU2006 整数程序指令分布特征:



SPEC CPU: 一套基准测试程序

SPEC CPU指令特征

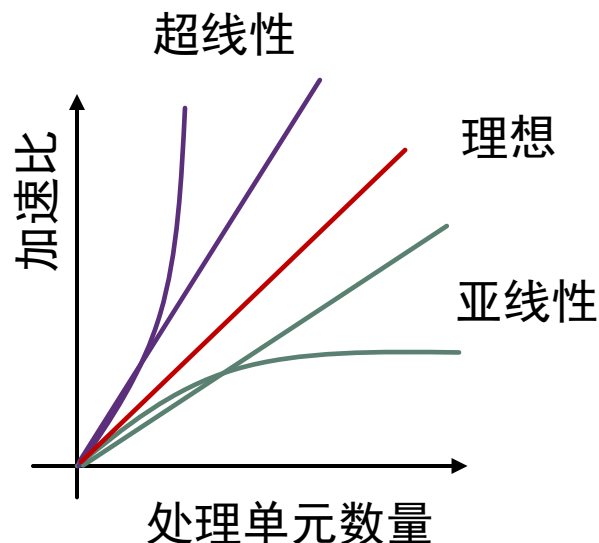
SPEC CPU2006 浮点程序指令分布特征:



并行性能指标

加速比

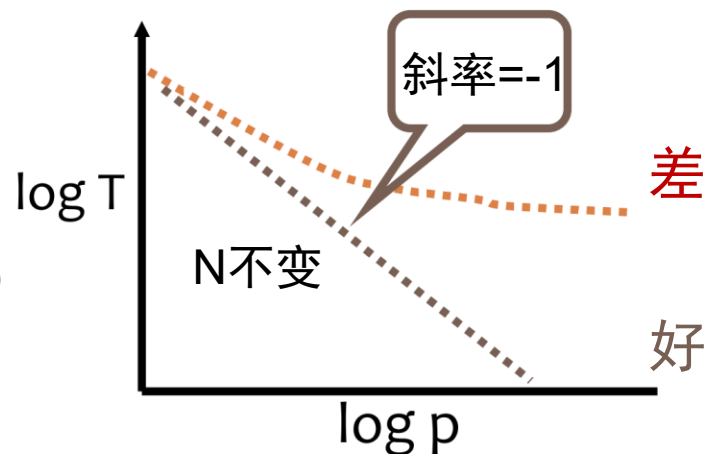
- **加速比**: $S(p) = \frac{T_S}{T_p}$
 - T_S : 程序的串行版本执行时间
 - T_p : 程序的并行版本执行时间, 使用的处理单元数目为 p
- **线性加速**: $S(p) = p$
 - 理论中的理想最大加速比
- **超线性加速**: $S(p) > p$
 - 实际中有时发生
 - 例如: 数据缓存在cache
- **亚线性加速**:
 - 典型加速情况



性能指标 – 并行程序扩展性

■ 强扩展性：

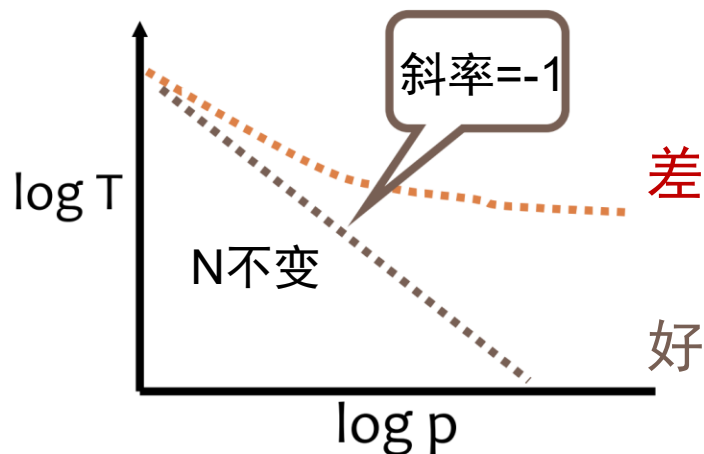
- 问题规模 (N) 不变
- 目标为更快运行同一个问题
- 完美扩展的运行时间为 $\frac{1}{p}$ （相对于串行）



性能指标 – 并行程序扩展性

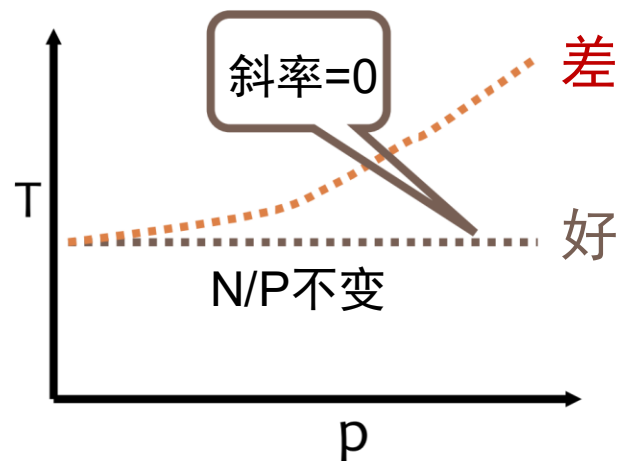
■ 强扩展性：

- 问题规模 (N) 不变
- 目标为更快运行同一个问题
- 完美扩展的运行时间为 $\frac{1}{p}$ (相对于串行)



■ 弱扩展性：

- 每个处理器的问题规模不变
- 目标为跑更大规模的问题，用同样的时间
- 完美扩展的运行时间保持不变

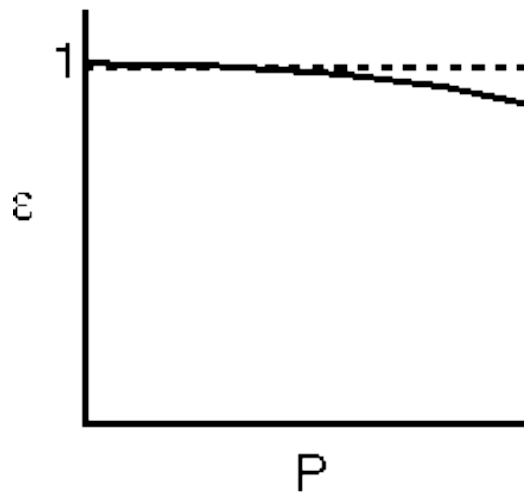


性能指标 – 并行效率

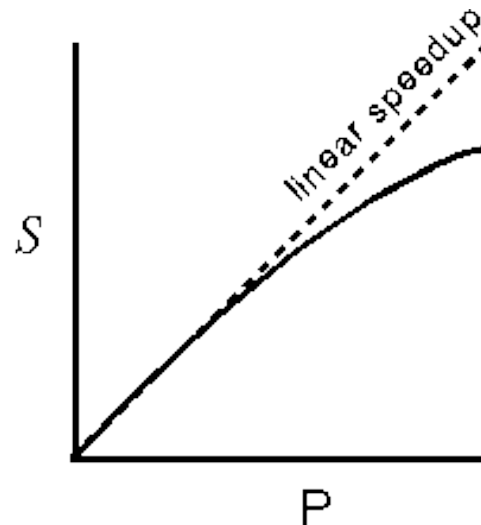
■ 并行效率 - Parallel Efficiency:

- P: 处理器数量
- T_1 : 最优的串行执行时间
- T_P : P 个处理器上执行时间
- N: 问题规模

$$e(N, P) = \frac{1}{P} \frac{T_1}{T_P}$$



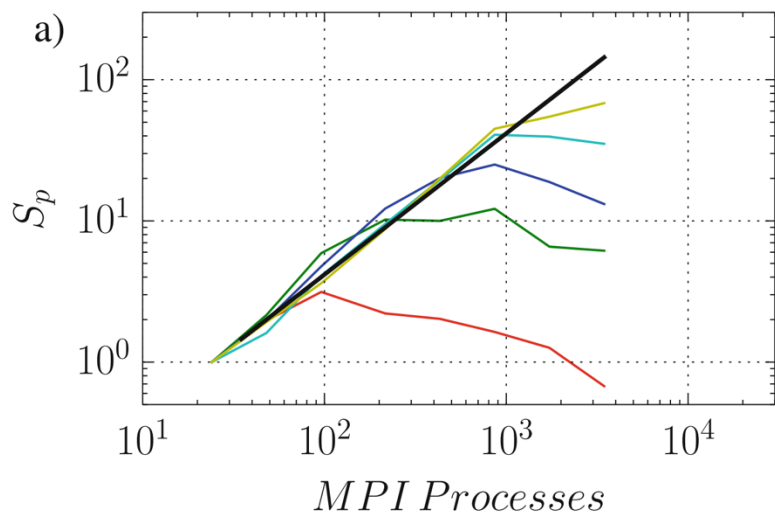
典型并行效率曲线



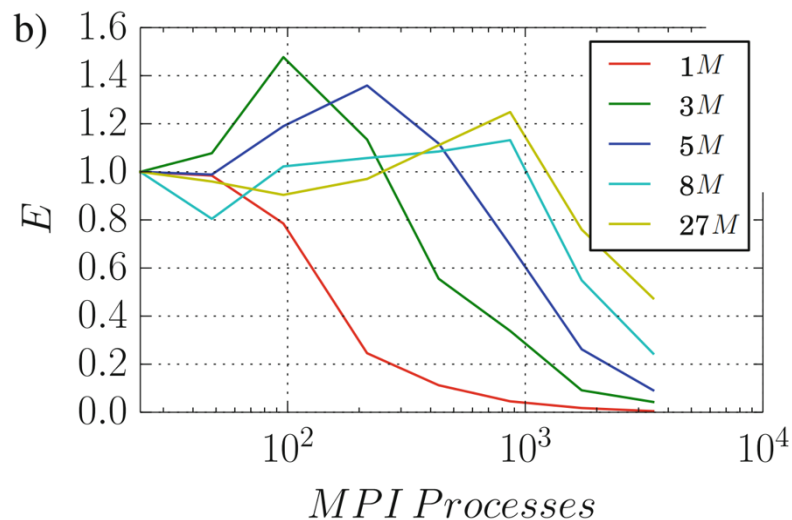
典型加速比曲线

并行程序扩展性示例

- 某计算流体力学程序的性能测试结果图
 - 颜色表示不同的问题规模



加速比曲线



并行效率曲线

程序热点

- **程序热点**（Hot spot）：程序运行时间较长函数或代码段
- 程序热点是**性能优化的关键**

Function / Call Stack	CPU Time ▼
▶ multiply1	352.577s
▶ init_arr	0.040s
▶ WaitForMultipleObjects	0.025s
▶ free	0.008s



53	for(i=tidx; i<msize; i=i+numt) {	
54	for(j=0; j<msize; j++) {	0.016s
55	for(k=0; k<msize; k++) {	2.183s
56	c[i][j] = c[i][j] + a[i][k] * b[k][j];	348.114s
57	}	2.265s
58	}	
59	}	
60	}	

矩阵乘的热点分析

性能分析步骤

1. 串行性能优化：

- 编译优化
- 体系结构优化
- 性能工具（Gprof、Perf、vTune）

2. 并行性能优化：

- 多线程、多进程相关问题（vTune）
- 通信（IPM、ITAC）
- I/O（iostat、vTune）
- 负载均衡（IPM、vTune）
- 每一次优化后，保留数据和代码，总结并思考下一步优化

关于程序性能的疑问

- 如何判断一个程序的性能好坏？

- 没有简单的标准，但可以检查：

- 扩展性

- 负载均衡

- 通信时间占比

- IO时间占比

- $\frac{\text{实际性能}}{\text{硬件理论性能}} > X\%$

取决于硬件和应用，如矩阵乘可以优化到90%以上

性能分析

—典型分析技术

性能分析技术

- 常见性能分析技术
 - 程序剖面 Profiling
 - 事件轨迹 Tracing
 - 插装 Instrumentation
 - 采样 Sampling
 - 硬件计数器 PMU

分析技术 – Profiling 与 Tracing

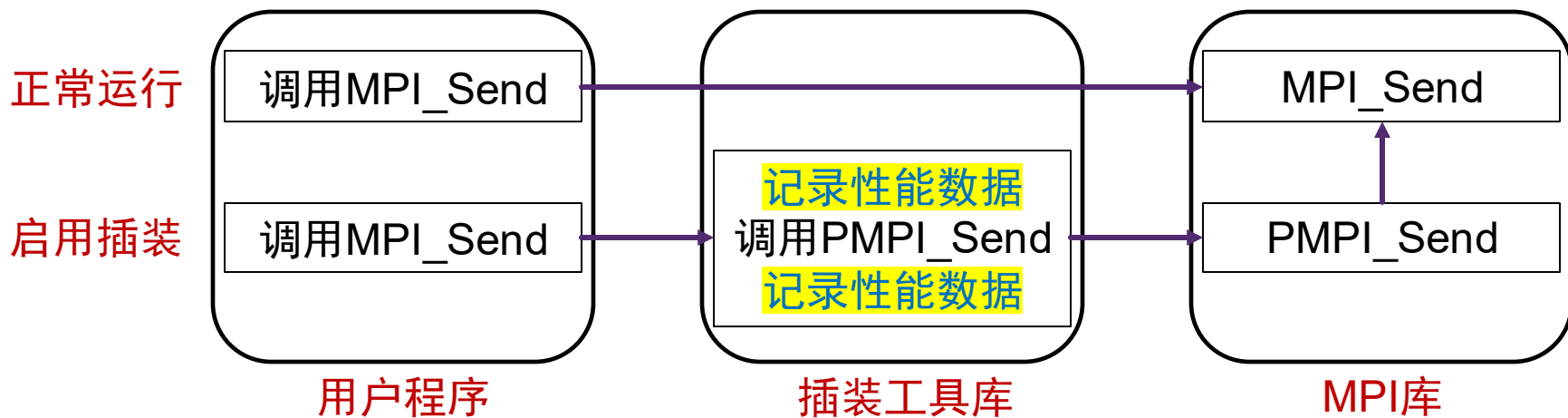
- **程序剖面 (Program Profiling)**
 - 一个应用程序执行行为的**统计信息**
 - 获取的性能数据叫 **profile**
 - **数据比较粗略**
 - 例如：程序每个函数执行时间分布、指令类型分布
- **事件轨迹 (Event Tracing)**
 - 按照程序执行的**时间顺序**记录每条事件详细信息
 - 获取的性能数据叫 **trace**
 - **数据比较详细**
 - 例如：程序的内存访问序列、指令访问序列

Profiling 与 Tracing 示例

- 例子：人事部门统计销售人员小张 8 月份的出差信息
- 程序剖面 (Profile):
 - 武汉 3 天，长沙 2 天，哈尔滨 2 天
- 事件轨迹 (Trace):
 - 8月5日 (周五) 18:00 - 8月7日 (周日) 21:00 长沙
 - 8月13日 (周六) 18:00 - 8月16 (周二) 22:00 武汉
 - 8月20日 (周六) 20:00 - 8月22 (周一) 19:00 哈尔滨

插装 Instrumentation

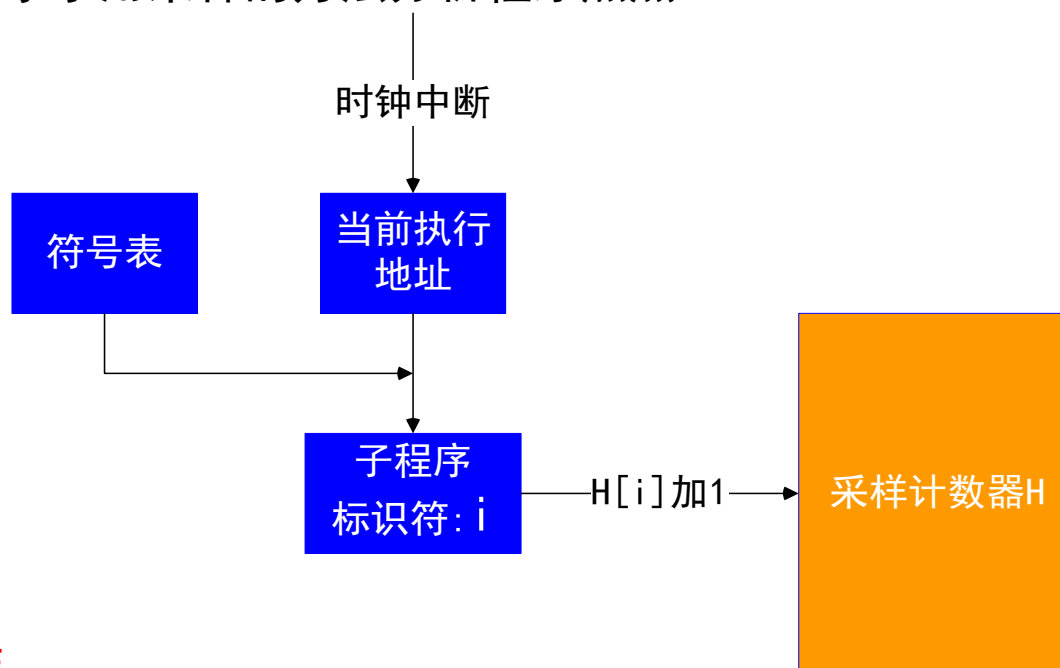
- **插装**：在待分析的代码块前后**插入分析代码**（例如：计时）
 - 基于**程序源代码**的插装
 - 基于**二进制文件**的插装
 - 库预留的性能分析接口：便于统计每个函数的耗时、事件轨迹
 - MPI 的 PMPI, OpenMP 的 OMPT, OMPD
 - 其它方法：二进制文件重写、指令中断



MPI 程序插装示例

采样

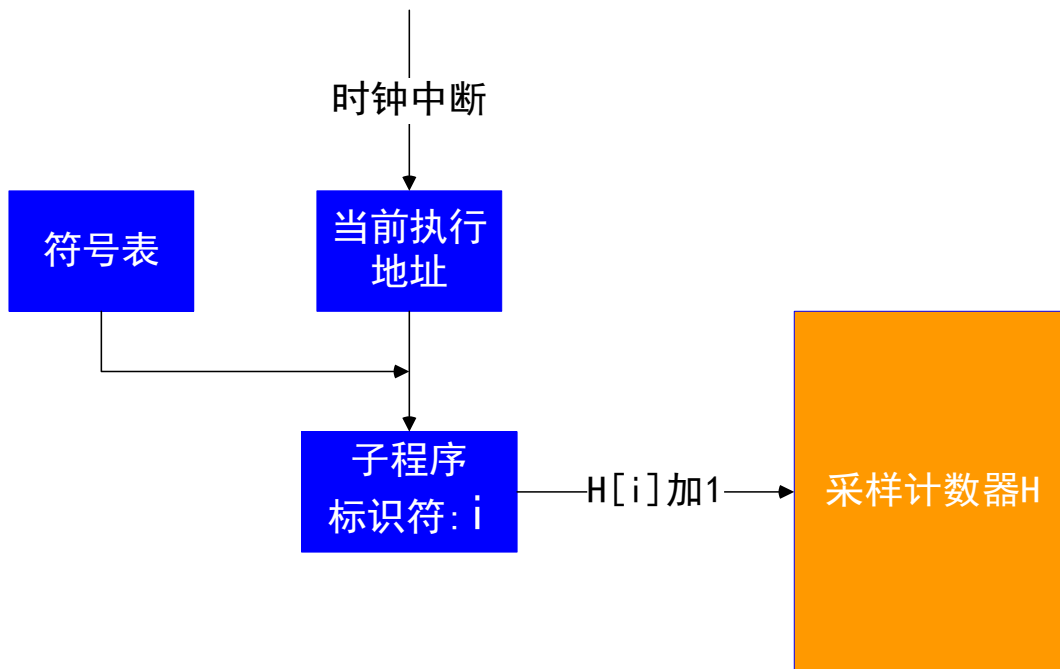
- **性能开销**和**存储开销**是影响性能工具可用性的重要因素
- **时间采样**
 - 每经过固定时间间隔，采集一次性能数据
 - 下图展示了用采样的方法分析程序热点



- **事件采样**
 - 当被观测的事件发生固定次数，采集一次性能数据
 - 常见的例子为 PMU 可统计的事件，如缓存缺失

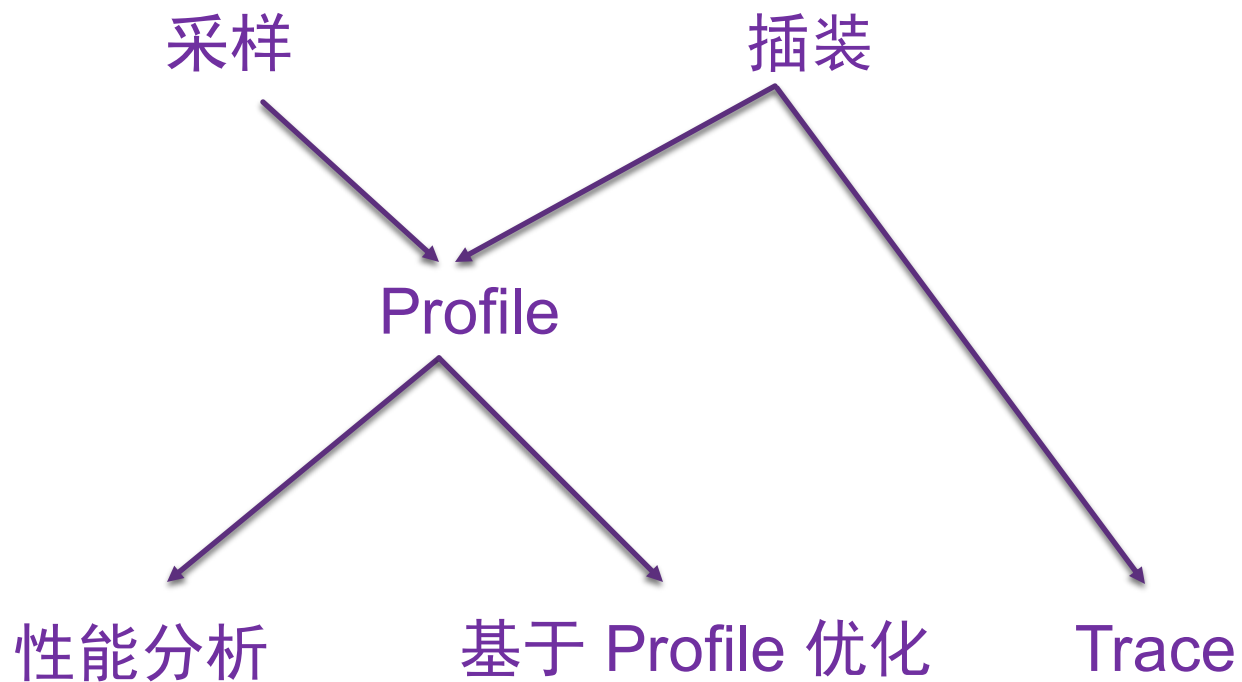
采样的例子

- 假设每 10ms 采样一次，程序执行时间为 8s。如果 $H[1] = 12$ ，那么子程序 P 占整个程序运行的时间比例是多少？



- $P = 12/800 = 0.015 = 1.5\%$

关系图



硬件性能计数器 PMU

■ 性能计数器（PMU, Performance Monitor Counter）

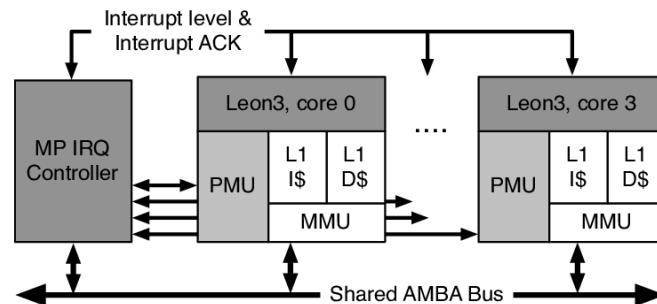
- 用于监测性能事件的**硬件寄存器**
- 采集相关性能指标，如：cache 缺失率等
- 帮助分析程序的性能特征、定位性能问题

■ 特点：

- 低开销
- 非侵入式、支持任意程序
- 可在**生产环境**中使用

■ 局限：

- 计数器**数量有限**：由硬件决定，常用的 Intel 处理器每个核 4 个左右
- **缺乏统一标准**：厂商间不同，不同代的产品间不同
- **直接使用复杂**：文档不完整



常见 PMU 指标

- **IPC**（每周期指令数，Instructions Per Cycle）或者 **CPI**（每指令周期数）

$$IPC = \frac{Total_Instructions_Exec.}{Total_Cycles}$$

- **L1 缓存命中率**：L1 中访问且命中的比例

$$1 - \frac{L1_Misses}{(Loads + Stores)}$$

- Intel平台L1缓存分为指令缓存（icache, instruction cache）与数据缓存（dcache, data cache），有独立的 PMU 统计信息
- **L2 缓存命中率**：L2 中访问（即 L1 没有命中）且命中的比例

$$1 - \frac{L2_Misses}{L1_Misses}$$

常见 PMU 指标

- 访存计算比:

$$\frac{(Loads + Stores)}{Flop_Inst_Exec.}$$

- 分支 (branch) 比率: 所有指令中分支指令的比率

$$\frac{Decoded_Branches}{Total_Instructions_Exec.}$$

- 分支预测错误率:

$$\frac{Mispredicted_Branches}{Total_Branches_Decoded}$$

- TLB 缺失率: 所有访存指令中, TLB 没有命中的比例

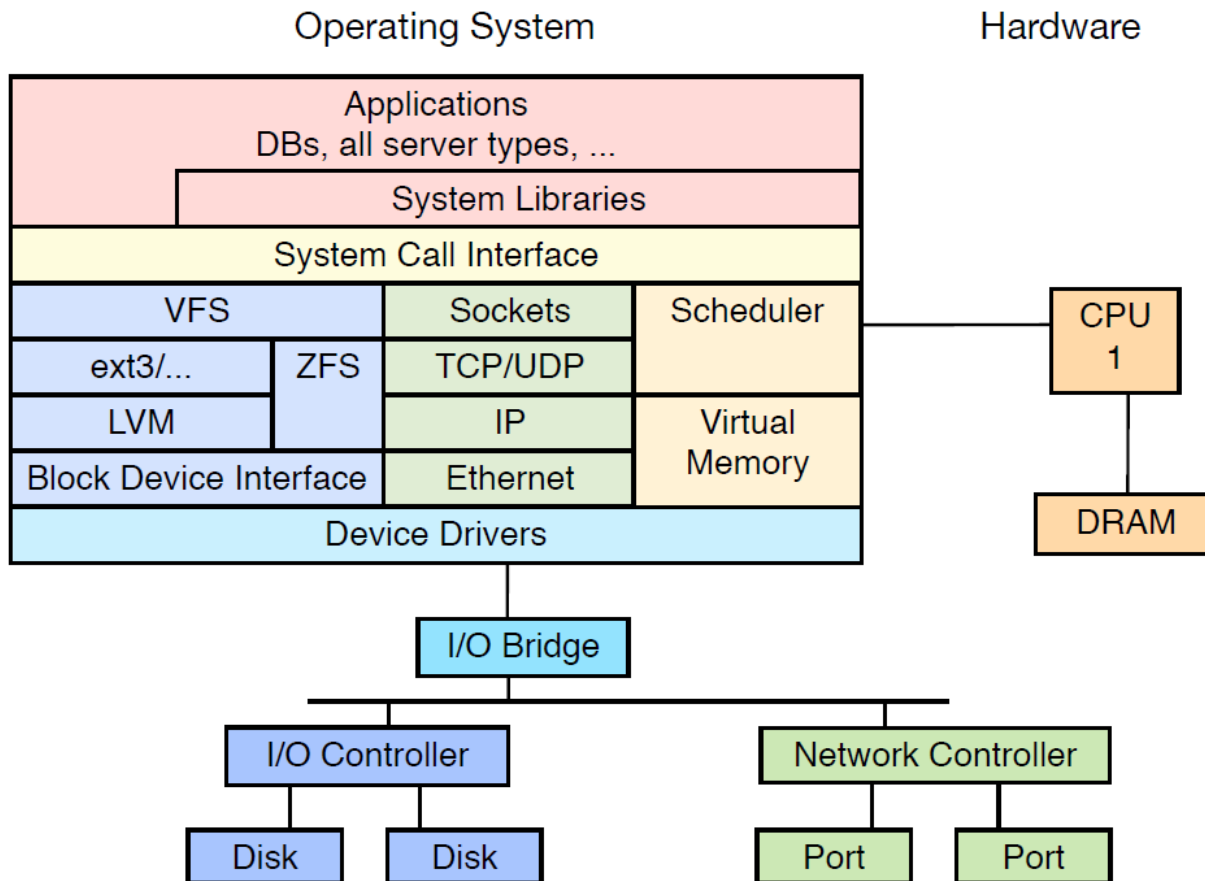
$$\frac{TLB_misses}{(Loads + Stores)}$$

性能分析

—典型分析工具

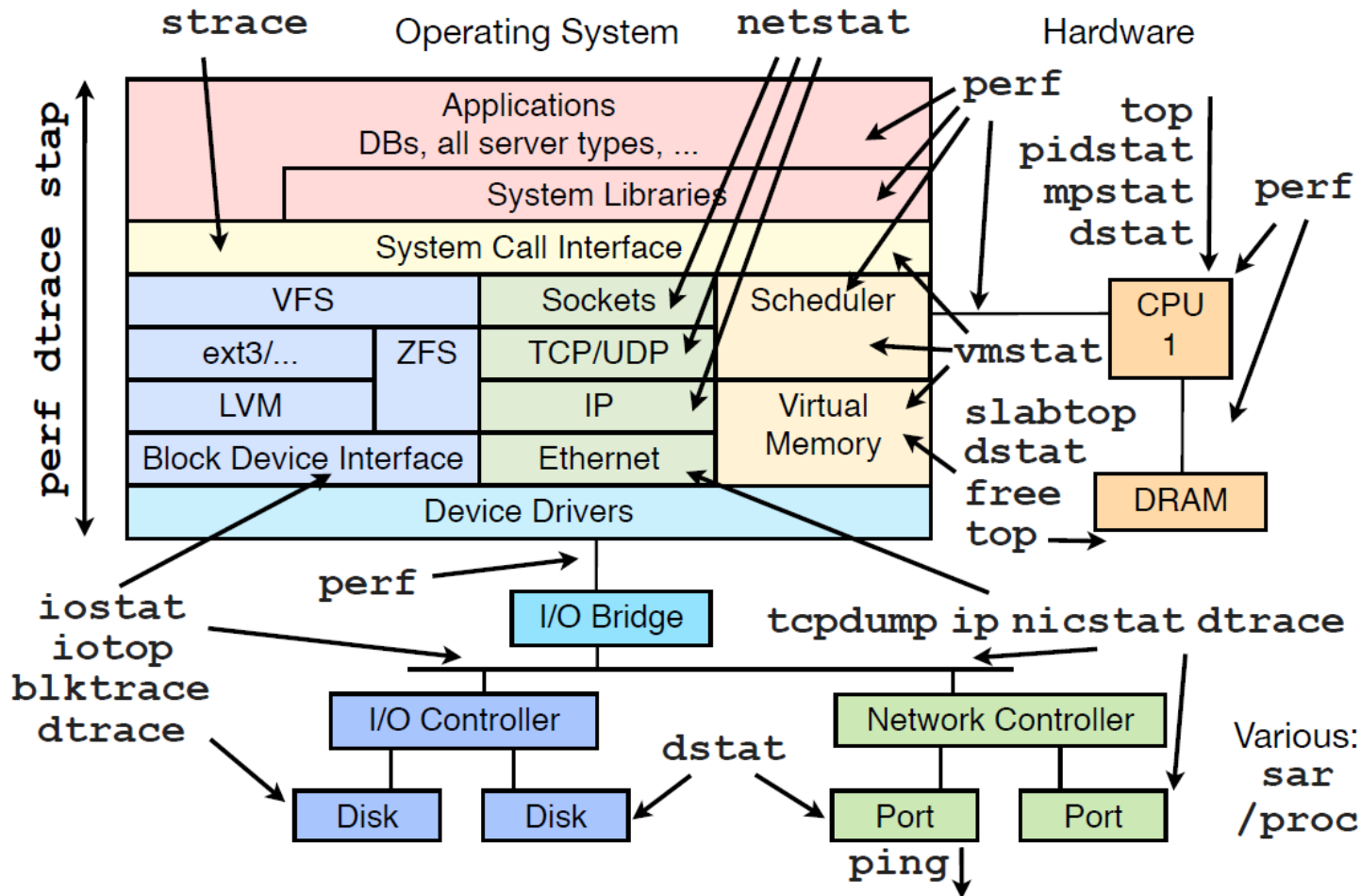
性能分析工具

■ 主要系统模块：



性能分析工具

■ 典型 Linux 性能工具:



GProf

- 在编译时：Gprof 插装每一个函数
- 在运行时：记录调用关系、函数运行时间
- 使用方法：
 - 在编译时用“-pg”编译选项启用 GProf
 - 运行程序
 - 运行 gprof 分析数据
- 输出了“每个函数自身”的用时（self）和“函数自身+调用的子函数”的用时（total）

```
user@host:~$ ./paraGraph
user@host:~$ # View profiling results with gprof
user@host:~$ gprof
```

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
69.35	0.43	0.43	1	430.00	430.00	build_incoming_edges(graph*)
30.65	0.62	0.19	18	10.56	10.56	pagerank(graph*, ...)
0.00	0.62	0.00	1632803	0.00	0.00	addVertex (VertexSet*, int)
0.00	0.62	0.00	7	0.00	0.00	newVertexSet(T, int, int)
0.00	0.62	0.00	7	0.00	0.00	freeVertexSet(VertexSet*)

Perf

- Linux 的一个性能分析工具集
 - 可统计 PMU 信息、热点等数据
 - 跨平台：x86, Arm, PowerPC64, UltraSPARC 等
 - 可分析单个应用或整个系统
- 通过 Perf 的子命令，可方便地统计PMU数据
 - `perf list` – 显示当前可用 PMU 统计指标
 - `perf stat` – 记录程序在一个时间段内（粗粒度）的 PMU 数据
 - `perf record` – 采样工具，可用于统计热点、程序内细粒度分析
 - `perf report` – 展示采样结果

Perf stat

- 可指定记录特定 PMU 信息
- `Perf stat [-e ...] [<command>]`
 - `-e, --event <event>` 要记录的 PMU 事件，不指定则使用默认事件
 - `<command>` 程序命令，不指定则监测整个系统
- 物理上的 PMU 计数器数量有限，记录过多事件需要复用（multiplexing）计数器
 - 复用（时间维度的采样）会降低分析准确性
 - 常用 CPU 提供至少 4 个 PMU 硬件计数器

Perf stat 输出

```
user@host:~$ perf stat -B dd if=/dev/zero of=/dev/null count=1000000
512000000 bytes (512 MB) copied, 0.956217 s, 535 MB/s
```

Performance counter stats for 'dd if=/dev/zero of=/dev/null count=1000000':

5,099 cache-misses	#	0.005 M/sec	(scaled from 66.58%)
235,384 cache-references	#	0.246 M/sec	(scaled from 66.56%)
9,281,660 branch-misses	#	3.858 %	(scaled from 33.50%)
240,609,766 branches	#	251.559 M/sec	(scaled from 33.66%)
1,403,561,257 instructions	#	0.679 IPC	(scaled from 50.23%)
2,066,201,729 cycles	#	2160.227 M/sec	(scaled from 66.67%)
217 page-faults	#	0.000 M/sec	
3 CPU-migrations	#	0.000 M/sec	
83 context-switches	#	0.000 M/sec	
956.474238 task-clock-msecs	#	0.999 CPUs	

直接的PMU数据

衍生的性能数据

0.957617512 seconds time elapsed

- 这个测例的潜在性能瓶颈是什么？

Perf top

- 查看系统中各程序的CPU使用情况
 - 提供比 top 更详细的CPU使用信息

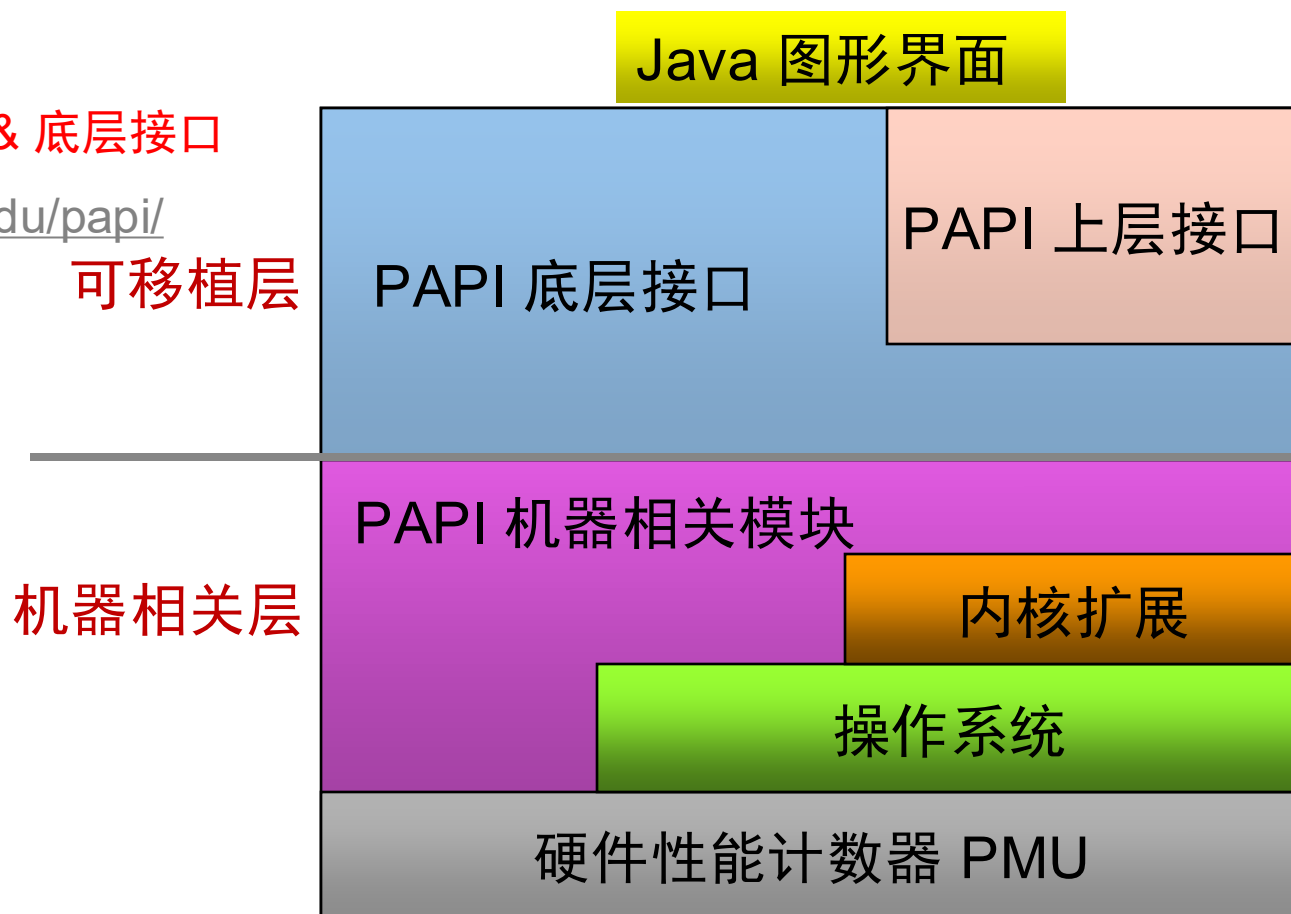
```
user@host:~$ perf top
```

```
Samples: 1K of event 'cycles:ppp', 2000 Hz, Event count (approx.): 614423613
```

Overhead	Shared Object	Symbol
12.61%	libc-2.28.so	[.] __uflow
10.03%	libc-2.28.so	[.] _IO_default_uflow
7.26%	install	[.] sigint_handler
5.73%	libc-2.28.so	[.] __printf_chk
4.63%	libc-2.28.so	[.] __strcasecmp_l_avx
4.33%	libc-2.28.so	[.] _IO_getc
4.23%	libc-2.28.so	[.] vfprintf
4.03%	libc-2.28.so	[.] _IO_file_underflow@@GLIBC_2.2.5
3.70%	python3.7	[.] _PyEval_EvalFrameDefault
2.55%	[kernel]	[k] smp_call_function_single
2.42%	install	[.] fgetc@plt
1.48%	python3.7	[.] _PyObject_GetMethod
1.33%	libc-2.28.so	[.] __memmove_avx_unaligned_erms
1.01%	libc-2.28.so	[.] _IO_file_xsputn@@GLIBC_2.2.5

PAPI

- PMU 的挑战：使用复杂
- PAPI：性能应用编程接口
 - 提供了跨平台的 PMU 编程接口
 - 方便使用 PMU
 - 提供上层接口 & 底层接口
 - <https://icl.utk.edu/papi/>



| PAPI

- papi_avail: 输出本地的可用 PAPI 事件信息

```
user@host:~$ papi_avail
Available PAPI preset and user defined events plus hardware information.
-----

PAPI version                : 5.6.0.0
Model string and code       : Intel(R) Xeon(R) CPU E5-2670 v3 @ 2.30GHz
(63, 0x3f)
Number Hardware Counters   : 10
-----

  PAPI Preset Events
    Name          Code      Avail Deriv Description (Note)
PAPI_L1_DCM      0x80000000   Yes   No   Level 1 data cache misses
PAPI_L1_ICM      0x80000001   Yes   No   Level 1 instruction cache misses
...
Of 108 possible events, 56 are available, of which 12 are derived.
```

PAPI 上层接口

- `PAPI_start_counters (int *events, int len)`
 1. 初始化 PAPI（若需要）
 2. 根据事件建立事件集合（event set）
 3. 开始统计事件集合中的事件
- `PAPI_stop_counters (long_long *vals, int alen)`
 - 停止计数并将结果保存在 vals 中
- `PAPI_accum_counters (long_long *vals, int alen)`
 - 将计数器累加到 vals 上，并重置计数器
- `PAPI_read_counters (long_long *vals, int alen)`
 - 将计数器的结果保存至 vals 中，并重置计数器

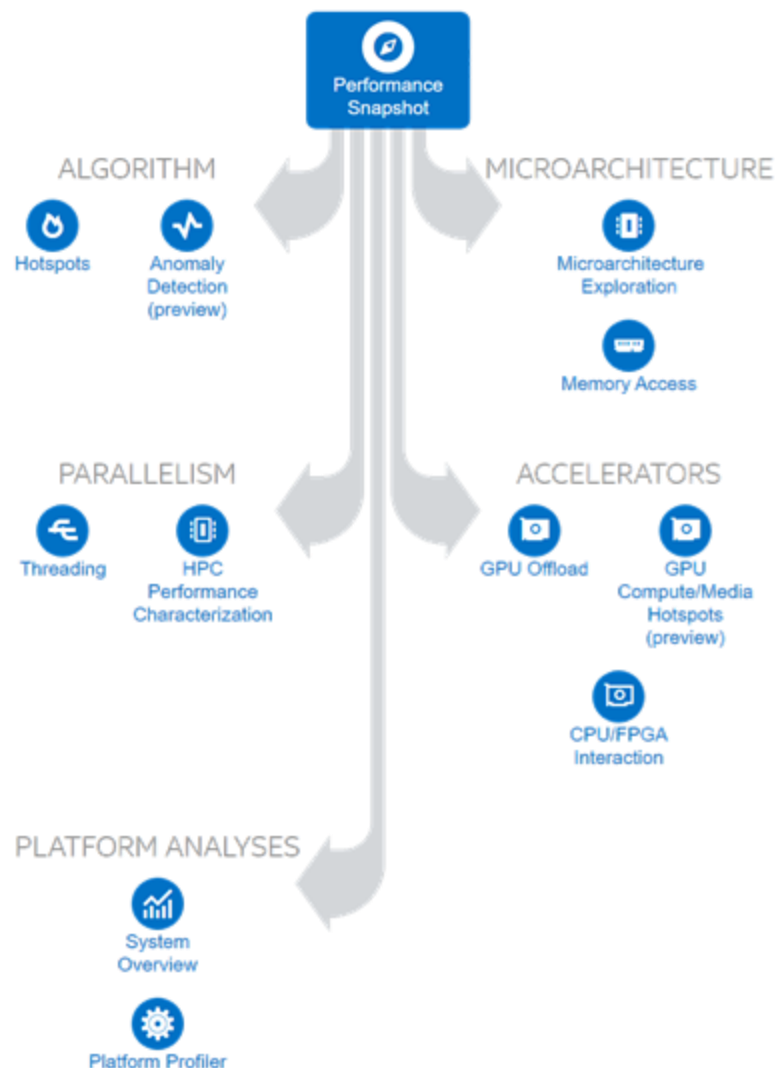
PAPI 上层接口示例

- 采用 PAPI 上层接口收集程序的性能数据

```
long long values[NUM_EVENTS];  
unsigned int Events[NUM_EVENTS]={PAPI_TOT_INS, PAPI_TOT_CYC};  
/** 开始计数器 **/  
PAPI_start_counters((int*)Events, NUM_EVENTS);  
/** 待分析的程序片段 **/  
computation_code();  
/** 停止计数器并把结果存储到相应变量 **/  
retval = PAPI_stop_counters(values, NUM_EVENTS);
```

vTune

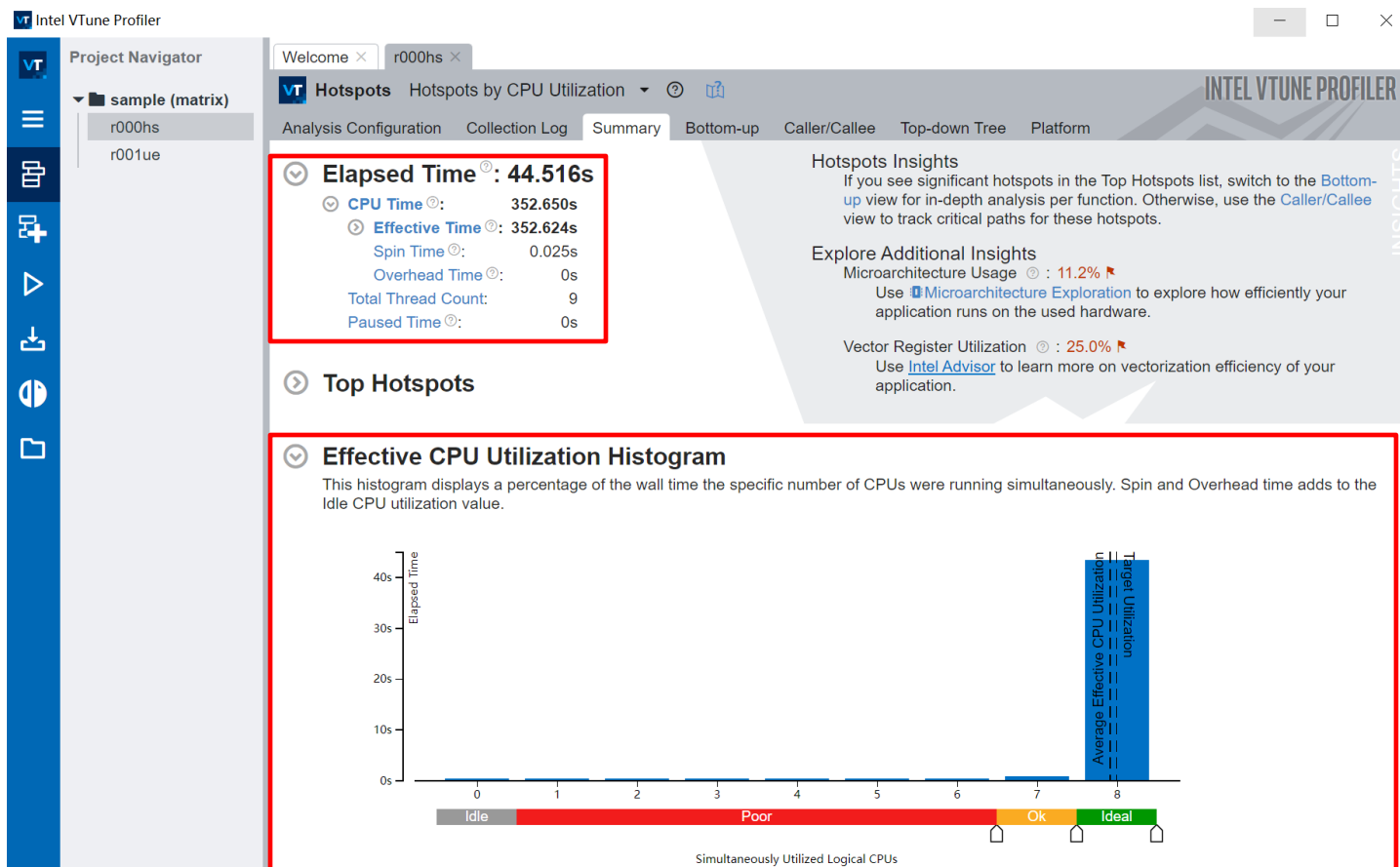
- **Intel 设备**上功能齐全的性能工具
 - 支持多线程、MPI 程序
 - 支持 Java, C# 等
 - 不需要重新编译程序
 - 良好的图形界面和数据可视化
- **典型分析功能**
 - 程序热点检测
 - 微体系架构性能指标
 - 访存分析
 - 并行性分析
 - 加速器性能分析



vTune 性能功能

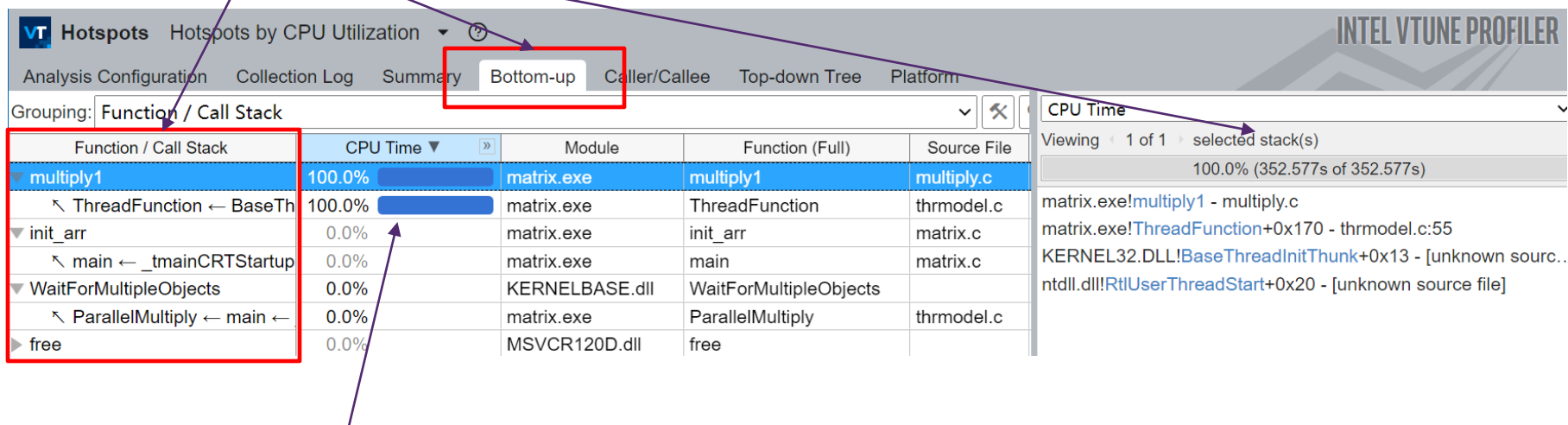
vTune – CPU 利用率分析

- CPU 利用率: $\frac{\text{CPU time}}{\# \text{CPU} \times \text{Wall time}}$ ($\frac{\text{程序使用的CPU核时}}{\text{CPU数} \times \text{经过时间}}$)
- 分析程序是否大部分时间都用在处理器



vTune - 热点分析

- Bottom-up视图：显示了该函数的调用关系（调用者信息）



- 关注耗时大的函数

vTune - 热点分析

■ 多个调用者的情况

VT Hotspots Hotspots by CPU Utilization ?				
Analysis Configuration Collection Log Summary Bottom-up Caller/Callee Top-down Tree				
Grouping: Function / Call Stack				
Function / Call Stack	CPU Time			
	Effective Time by Utilization Idle Poor Ok Ideal Over	Spin Time	Overhead Time	
grid_intersect	4.575s	0s	0s	
▶ intersect_objects	4.171s	0s	0s	
▶ grid_intersect ← intersect	0.403s	0s	0s	
▶ sphere_intersect	3.590s	0s	0s	
▶ func@0x69e19df0	2.491s	0s	0s	
▶ PeekMessageA	1.510s	0s	0s	
▶ GdipDrawImagePointRectI	1.245s	0s	0s	
▶ RtlEnterCriticalSection	0s	1.182s	0s	

vTune - 热点分析

- 细粒度热点分析
 - 将性能数据对应至源代码和汇编代码

VT

Hotspots

Hotspots by CPU Utilization

Analysis Configuration

Collection Log

Summary

Bottom-up

Caller/Callee

Top-down Tree

Platform

multiply.c

Source

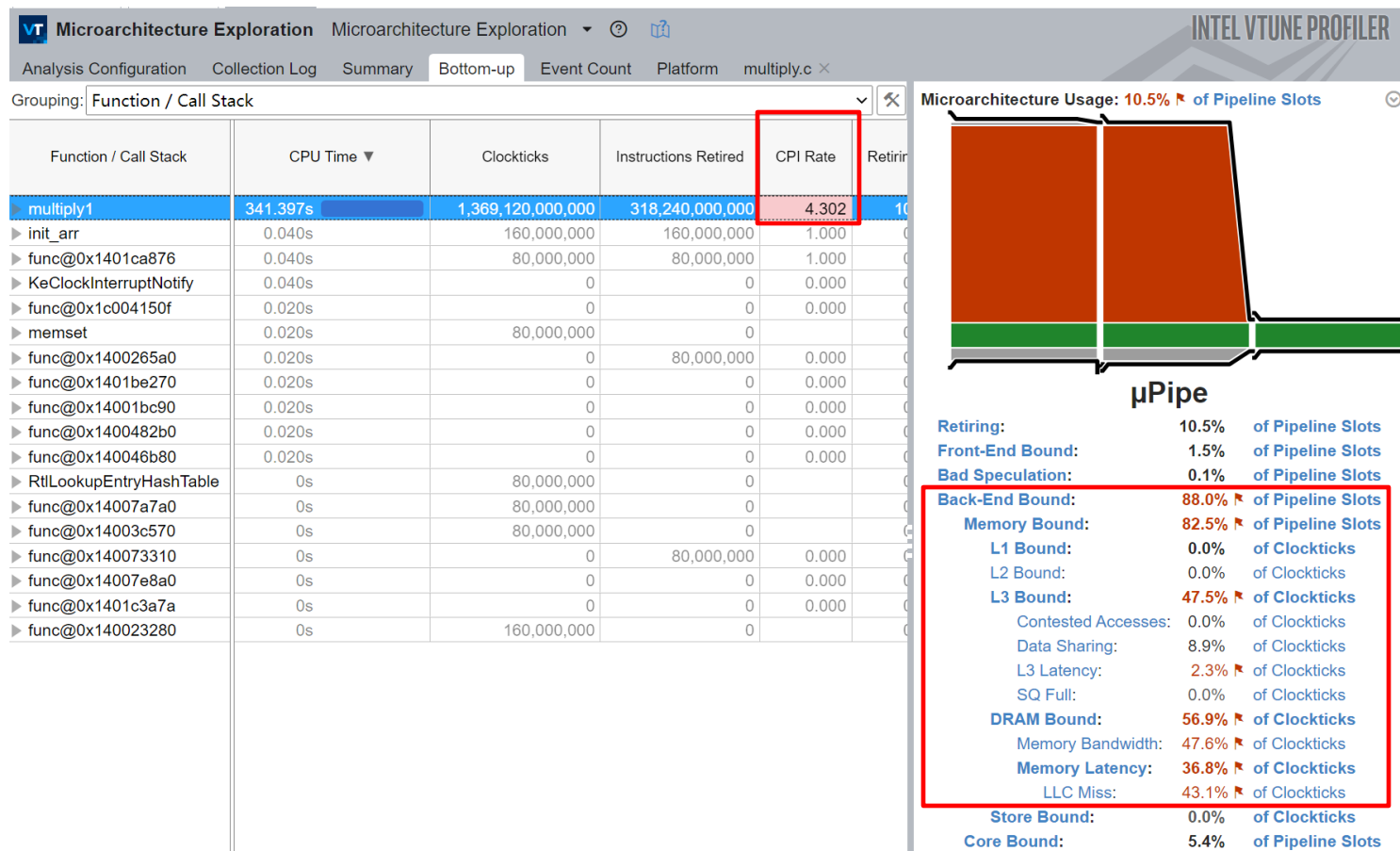
Assembly

Assembly grouping: Address

S...	Source	CPU Tim...	CPU Time: Self	Address	So...	Assembly	CPU Time: Total	CPU Time: Self
42				0x140001b14	55	mov eax, dword ptr [rsp+0x2]	0.6%	2.110s
43	void multiply1(int msize, int tid, int numt, TYPE a[][NUM])			0x140001b18	55	cmp dword ptr [rsp+0x8], ea	0.0%	0.041s
44	{			0x140001b1c	55	jmp 0x140001bb1 <Block 12>		
45	int i,j,k;			0x140001b22		Block 11:		
46				0x140001b22	56	movsxd rax, dword ptr [rsp]	0.0%	0.078s
47	// This naive implementation of matrix multiply contains an			0x140001b26	56	imul rax, rax, 0x4000		
48	// Each iteration of the inner loop strides across the full			0x140001b2d	56	mov rcx, qword ptr [rsp+0x4]	0.6%	1.966s
49	// because the iterator 'k' is used in the first dimension			0x140001b32	56	add rcx, rax	0.0%	0.025s
50	// This leads to bad cache reuse and significant memory sta			0x140001b35	56	mov rax, rcx		
51	// Use Microarchitecture and Memory access analysis to est			0x140001b38	56	movsxd rcx, dword ptr [rsp+		
52	// See the 'multiply2' function implementation to overcome			0x140001b3d	56	movsxd rdx, dword ptr [rsp]	0.5%	1.873s
53	for(i=tidx; i<msize; i=i+numt) {			0x140001b41	56	imul rdx, rdx, 0x4000	0.0%	0.031s
54	for(j=0; j<msize; j++) {	0.0%	0.016s	0x140001b48	56	mov r8, qword ptr [rsp+0x38]		
55	for(k=0; k<msize; k++) {	0.6%	2.183s	0x140001b4d	56	add r8, rdx	0.0%	0.016s
56	c[i][j] = c[i][j] + a[i][k] * b[k][j];	98.7%	348.114s	0x140001b50	56	mov rdx, r8	0.6%	2.075s
57	}	0.6%	2.265s	0x140001b53	56	movsxd r8, dword ptr [rsp+0	0.0%	0.047s
58	}			0x140001b58	56	movsxd r9, dword ptr [rsp+0		
59	}			0x140001b5d	56	imul r9, r9, 0x4000		
60	}			0x140001b64	56	mov r10, qword ptr [rsp+0x4	0.6%	2.147s
61				0x140001b69	56	add r10, r9	0.0%	0.062s
62	void multiply2(int msize, int tid, int numt, TYPE a[][NUM])			0x140001b6c	56	mov r9, r10	0.0%	0.031s
63	{			0x140001b6f	56	movsxd r10, dword ptr [rsp+		
64	int i,j,k;			0x140001b74	56	movsd xmm0, qword ptr [rdx+	0.6%	2.110s
65				0x140001b7a	56	mulsd xmm0, qword ptr [r9+r	0.1%	0.345s
66	// This implementation interchanges the 'j' and 'k' loop it			0x140001b80	56	movsd xmm1, qword ptr [rax+	92.2%	325.285s
67	// The loop interchange technique removes the bottleneck ca			0x140001b85	56	addsd xmm1, xmm0	0.5%	1.676s
68	// memory access pattern in the 'multiply1' function.			0x140001b89	56	movaps xmm0, xmm1	2.3%	8.127s
69	for(i=tidx; i<msize; i=i+numt) {			0x140001b8c	56	movsxd rax, dword ptr [rsp]	0.0%	0.047s
70	for(k=0; k<msize; k++) {			0x140001b90	56	imul rax, rax, 0x4000	0.1%	0.260s
71	for(j=0; j<msize; j++) {			0x140001b97	56	mov rcx, qword ptr [rsp+0x4		
72	c[i][j] = c[i][j] + a[i][k] * b[k][j];			0x140001b9c	56	add rcx, rax	0.5%	1.856s
73	}			0x140001b9f	56	mov rax, rcx	0.0%	0.042s
74	}			0x140001ba2	56	movsxd rcx, dword ptr [rsp+	0.0%	0.016s
75	}			0x140001ba7	56	movsd qword ptr [rax+rcx*8]		
76	}			0x140001bac	57	jmp 0x140001b0a <Block 9>	0.6%	2.285s
77				0x140001bb1		Block 12:		
78	void multiply3(int msize, int tid, int numt, TYPE a[][NUM])			0x140001bb1	58	jmp 0x140001ae8 <Block 6>		
79	{			0x140001bb6		Block 13:		
				0x140001bb6	59	jmp 0x140001ac3 <Block 3>		

vTune - 微体系架构分析

■ 此矩阵乘代码的性能问题来自什么？



访存是主要的性能瓶颈

IPM

- IPM (Integrated Performance Monitoring)
 - MPI 程序通信分析
 - 统计不同 MPI 函数的耗时，查看通信用时比例、热点通信函数
- 特点：
 - 性能开销小
 - 可用于分析不同进程的负载均衡情况
 - 不用重新编译程序：插装 MPI 库的 PMPI 接口实现数据采集
- 使用：
 - Intel MPI 集成了IPM，通过设置环境变量 `I_MPI_STATS=ipm` 即可
 - 其它 MPI，设置环境变量加载 libipm.so
 - `LD_PRELOAD = /path/to/ipm/lib/libipm.so mpirun ./a.out`

IPM 输出

```
#####
#
# command : /scratch/lbs (completed)
# host    : compute033/x86_64 Linux
# start   : 02/14/18/06:58:32
# stop    : 02/14/18/06:58:36
# gbytes  : 0.00000e+00 total
#
#####
# region : *      [ntasks] = 8
#
#
# [total]      <avg>      min      max
# entries      8          1          1          1
# wallclock    23.5715    2.94643   2.52573   4.22571
# user         465.128    58.141    57.9635   58.4132
# system       28.6847    3.58559   3.21678   3.89737
# mpi          7.27317    0.909146  0.45308   2.23711
# %comm        30.8558    17.8016   52.9404
# gflop/sec    NA        NA        NA        NA
# gbytes       0         0         0         0
#
# [time]      [calls]    <%mpi>    <%wall>
# MPI_Init_thread 5.00677    8        68.84    21.24
# MPI_Waitall     1.66641    5784     22.91    7.07
# MPI_Isend       0.249647   14400    3.43     1.06
# MPI_Barrier     0.0778501    8        1.07     0.33
# MPI_Allreduce   0.0718162    8        0.99     0.30
# MPI_Send        0.0708234    48       0.97     0.30
# MPI_Irecv       0.0533149   14448    0.73     0.23
# MPI_Bcast       0.0359943    24       0.49     0.15
# MPI_Scan        0.0223556    16       0.31     0.09
# MPI_Cart_create 0.0177999    8        0.24     0.08
# MPI_Finalize    0.000102282   8        0.00     0.00
# MPI_Cart_coords 0.000101328   56       0.00     0.00
# MPI_Cart_rank   5.91278e-05   96       0.00     0.00
# MPI_Cart_shift  5.36442e-05   24       0.00     0.00
# MPI_Comm_size   3.40939e-05   8        0.00     0.00
# MPI_Comm_rank   3.19481e-05   16       0.00     0.00
# MPI_Wtime       4.05312e-06   8        0.00     0.00
# MPI_TOTAL       7.27317      34968    100.00    30.86
#####
```

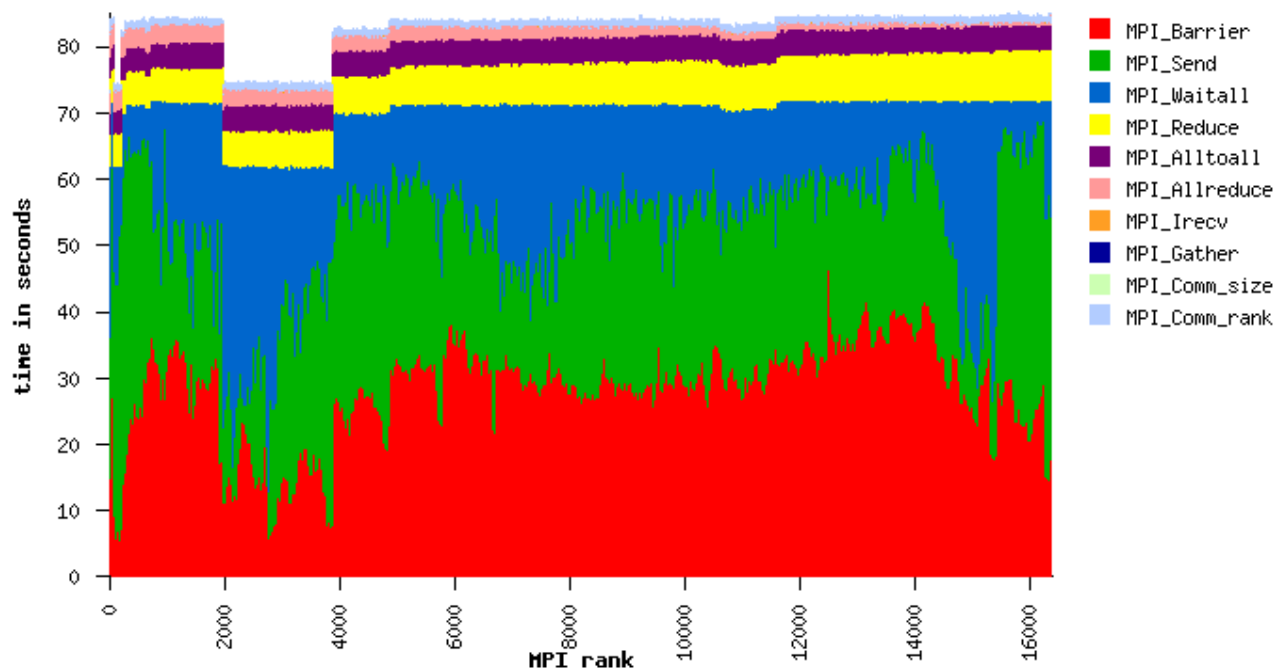
总体信息

程序耗时分布

MPI 通信耗时分布

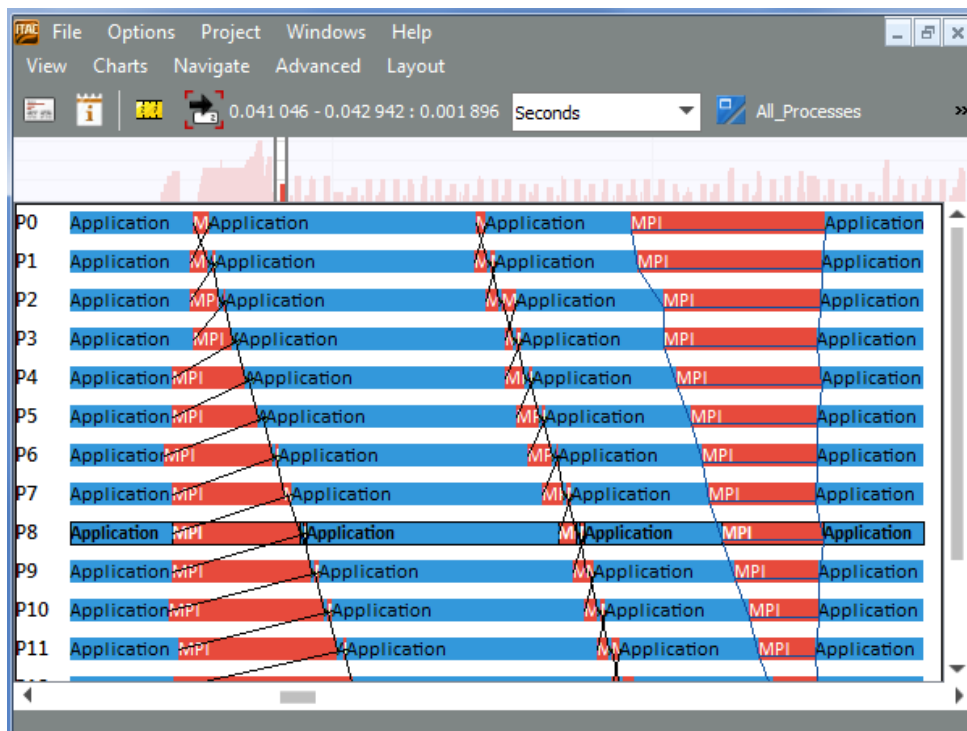
IPM 输出可视化

- 对各个进程的数据进行可视化
 - 不同进程的负载均衡
 - 关注 MPI_Waitall, MPI_Barrier



IPM 输出可视化示例

- Intel Trace Analyzer and Collector
- MPI 程序的事件轨迹（Trace）采集工具
 - 保留了时间维度的信息，可用于分析进程间的通信依赖
 - 红色为 MPI 耗时，黑线表示通信关系

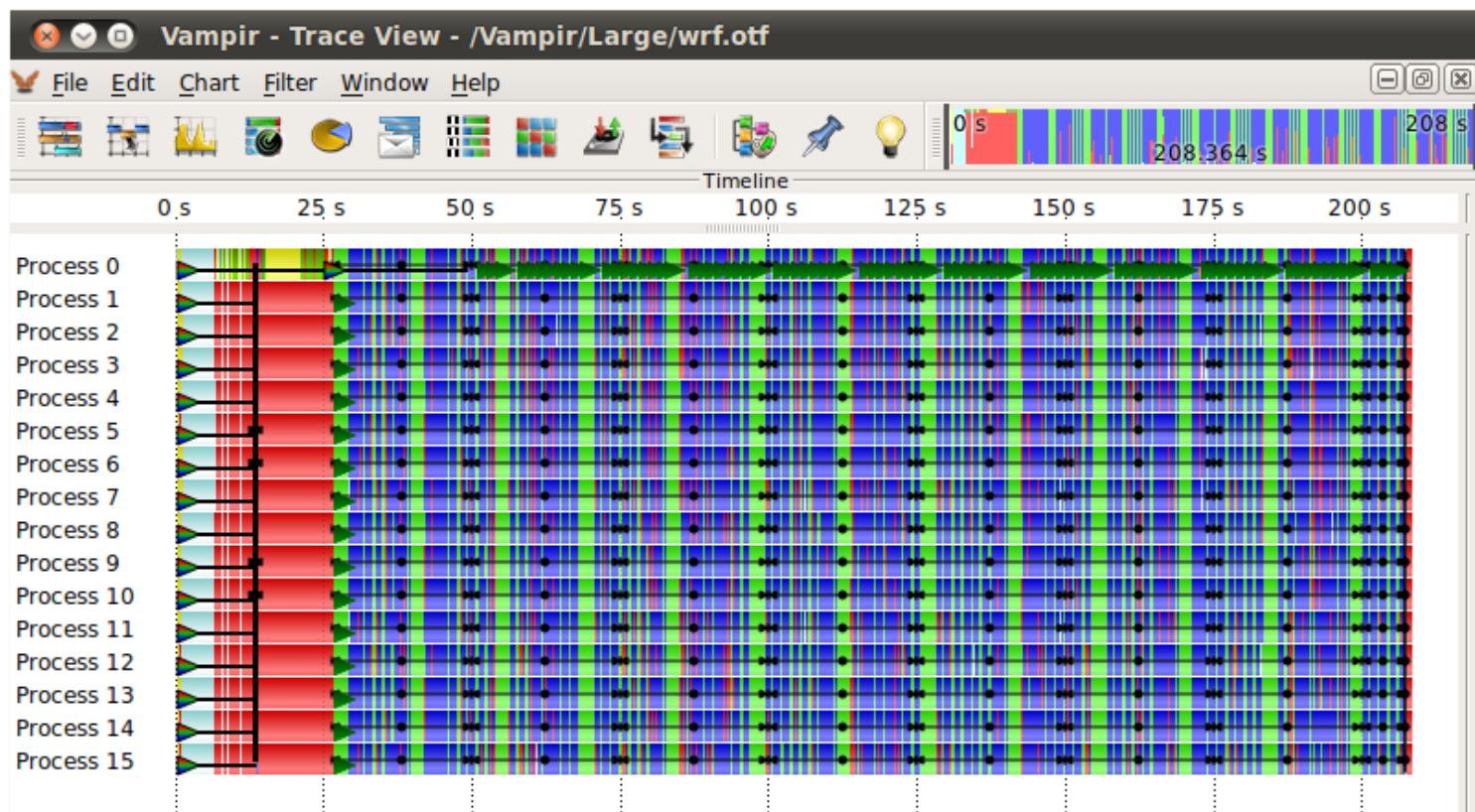


| ITAC使用方法

- Intel MPI运行时加 `-trace` 参数
 - `mpirun -trace -n 4 ./myApp`
- 运行后分析结果
 - `traceanalyzer ./myApp.stf`

Vampir 和 Vampirtrace

- 和 ITAC 类似
- 开源的 MPI Trace 采集和分析工具



PIN: 动态插装

- **PIN: 二进制代码动态插装工具**

- Intel 和 Virginia 大学开发
- 免费但不开源

- **基本原理:**

- 类似一个虚拟机
- 截获用户指令、插入额外指令、生成新指令执行
- 用户需要编写分析代码

- **支持插装粒度:**

- 指令
- 基本块
- Trace (自己定义一组基本块)

- **例子:**

- 输出程序每条访存操作: **地址、访问大小**



Pintool: 打印每条指令IP

```
Print(ip);  
sub    $0xff, %edx  
Print(ip);  
cmp    %esi, %edx  
Print(ip);  
jle    <L1>  
Print(ip);  
mov    $0x1, %edi  
Print(ip);  
add    $0x10, %eax
```

ManualExamples/itrace.cpp

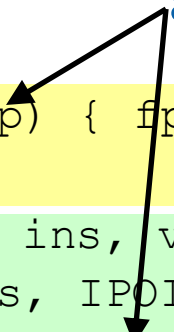
```
#include <stdio.h>
```

```
#include "pin.H"
```

```
FILE * trace;
```

```
void printip(void *ip) { fprintf(trace, "%p\n", ip); }
```

argument to analysis routine



analysis routine

```
void Instruction(INS ins, void *v) {
```

instrumentation routine

```
    INS_InsertCall(ins, IPOINT_BEFORE, (AFUNPTR)printip,
```

```
        IARG_INST_PTR, IARG_END);
```

```
}
```

```
void Fini(INT32 code, void *v) { fclose(trace); }
```

```
int main(int argc, char * argv[]) {
```

```
    trace = fopen("itrace.out", "w");
```

```
    PIN_Init(argc, argv);
```

```
    INS_AddInstrumentFunction(Instruction, 0);
```

```
    PIN_AddFiniFunction(Fini, 0);
```

```
    PIN_StartProgram();
```

```
    return 0;
```

```
}
```

| Pintool

```
$ pin -t itrace.so -- /bin/ls
```

```
Makefile imageload.out itrace proccount  
imageload inscount0 atrace itrace.out
```

```
$ head -4 itrace.out
```

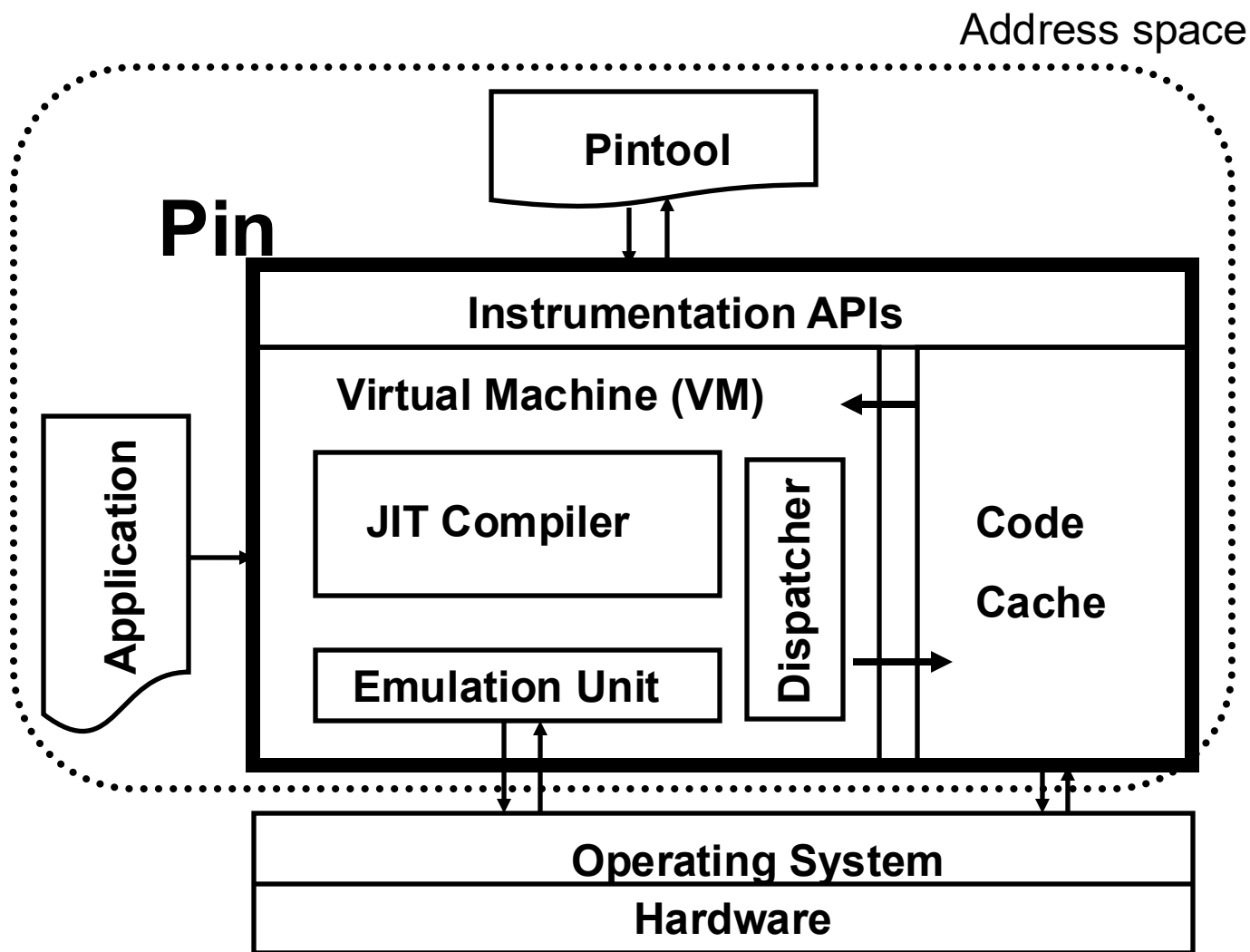
```
0x40001e90
```

```
0x40001e91
```

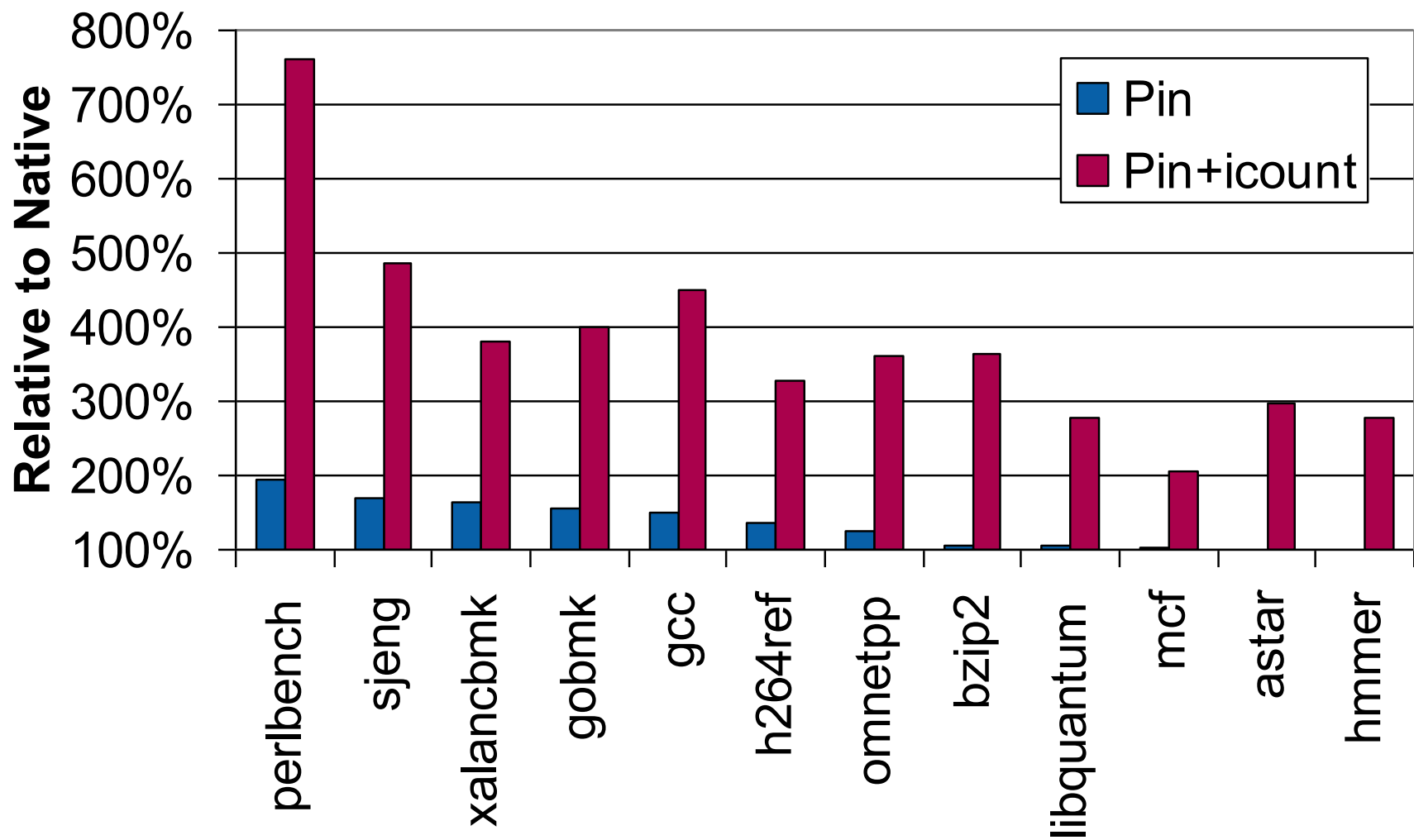
```
0x40001ee4
```

```
0x40001ee5
```

PIN工作原理



PIN开销



正确性调试

典型并程序 bug

- 典型并程序 bug
 - crash
 - hang（挂起、不能完成）
 - 多次执行得到不一致结果
 - 产生错误结果
 - 出现一些不期望行为

并程序正确性调试是一件有挑战的事情

常见程序 bug 原因

- **串行程序错误：**
 - 无效内存引用
 - 数组访问越界
 - 除以 0
 - 使用未初始化变量
- **并行程序错误：**
 - 未配对的发送和接收
 - 在发送前调用阻塞的接收
 - 数据竞争
 - 共享变量的错误修改
 - 死锁

串行错误和并行错交织到一起，使得调试更加有挑战

正确性调试

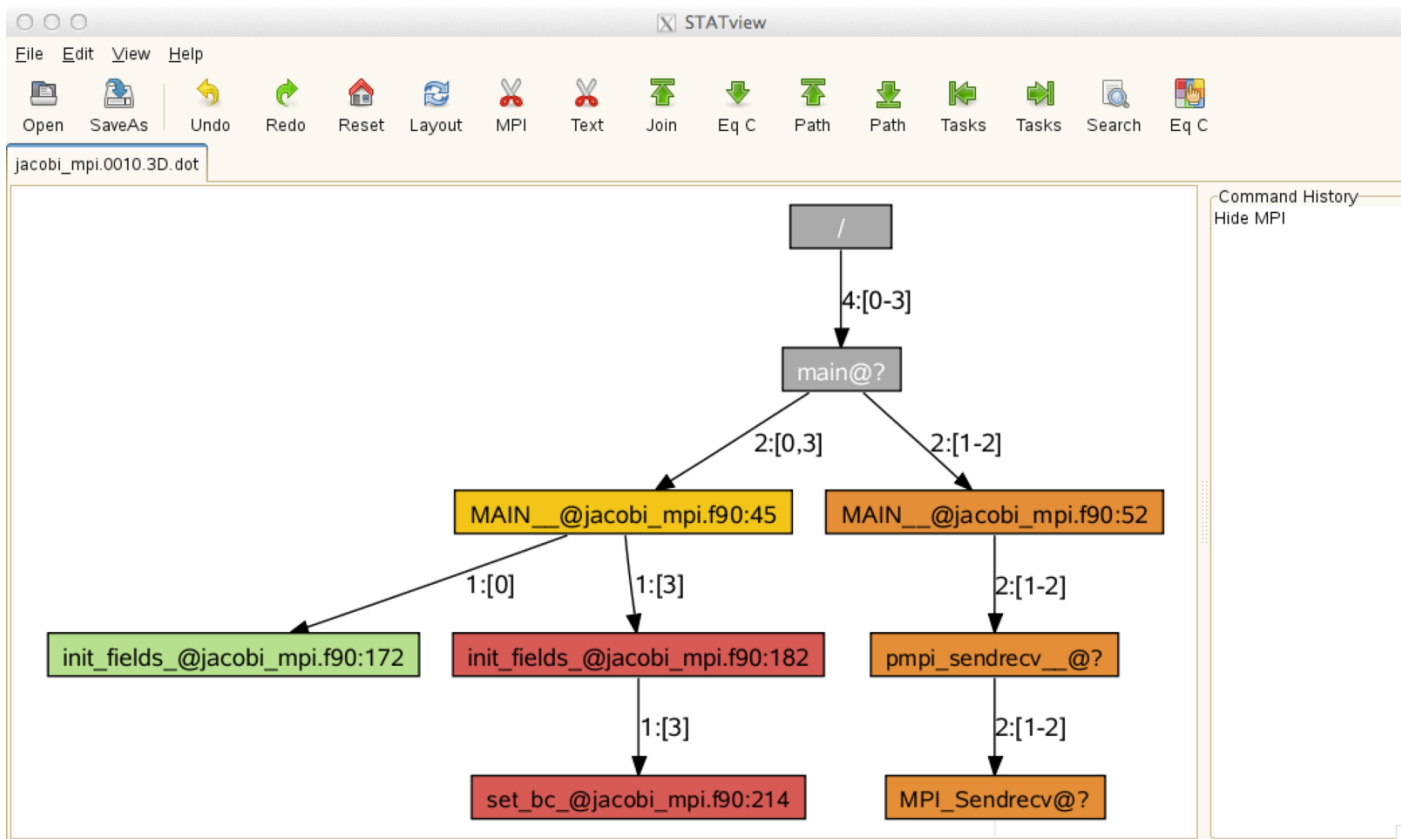
- **通用工具（手动，依靠程序员手动分析）**
 - printf：常用方法、不可扩展、不能交互
 - GDB：调试大量进程程序较为困难
 - 商业软件：Totalview，DDT（Distributed Debugging Tool）
- **专用工具（相对自动，工具直接定位问题）**
 - 常见错误：
 - 内存故障：重复释放、访问越界
 - 多线程 & MPI 故障：数据竞争、死锁、程序崩溃
 - 逻辑错误
 - 常用工具：
 - STAT
 - Valgrind
 - Sanitizer

STAT

- STAT (Stack Trace Analysis Tool)
 - MPI 程序调用栈分析工具
 - 收集各个 MPI 进程的函数调用栈中的函数调用关系
 - 通过调用树 (call tree) 来展示各个进程所在的位置
 - 可用于调试 hang 的 MPI 程序 bug
 - 在使用 STAT 后, 可结合 DDT 或 Totalview 进一步定位原因
 - 不适用于 OpenMP 程序
- ATP (Abnormal Termination Processing)
 - ATP 工具底层基于 STAT, 在程序崩溃时记录调用栈信息

STAT 可视化

- 把相似进程的信息进行汇总：



0号进程的信息

3号进程的信息

1、2号进程的信息

Valgrind 调试工具

- Valgrind:
 - 包括一组正确性调试和性能分析工具
- 分析工具:
 - memcheck: 访存错误和内存泄露检查
 - cachegrind: cache 和分支预测性能分析
 - callgrind: 基于调用关系图的 cache 和分支预测性能分析
 - massif, dhat: 堆的性能分析
 - Helgrind, drd: pthread 错误检测

Valgrind - memcheck

■ 可检测的问题：

- 使用未初始化的内存
- 访问未分配的、已释放的内存
- 内存泄漏

■ 基本原理：

1. 替换 C 内存分配器，在分配的地址块前后设置内存守护区（memory guard）
2. 插装程序的几乎所有指令，检查每个内存访问地址的有效性和数据有效性
 - 地址有效：已申请、未被释放的地址块
 - 数据有效：已初始化

■ 性能开销：

- 使得程序慢 20-30 倍
- 额外内存开销

Address Sanitizer 工具

- 基于编译的内存漏洞检查工具
 - GCC 和 LLVM 支持
 - 降低一定检测开销：性能开销约 2 倍
- 通过 `-fsanitize=address` 编译选项启用
 - `gcc -fsanitize=address -g a.c`
- 可检测的问题（与 Memcheck 类似）
 - 堆栈访问越界
 - 访问已释放空间
 - 栈溢出
 - 内存泄漏

Address Sanitizer 示例

```
1. /** overflow.c **/  
2. #include <stdio.h>  
3. int main(void) {  
4.     int a[3] = {0};  
5.     /** 非法内存访问 **/  
6.     a[3] = 1;  
7.     printf("%d\n", a[3]);  
}
```

```
user@host:~$ g++ -fsanitize=address -g ./overflow.c  
user@host:~$ ./a.out
```

```
=====
==224==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fffe28a4a1c at pc  
0x7f87d1200c0b bp 0x7fffe28a49e0 sp 0x7fffe28a49d0  
WRITE of size 4 at 0x7fffe28a4a1c thread T0  
#0 0x7f87d1200c0a in main overflow.c:6  
#1 0x7f87cfa61b96 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x21b96)  
#2 0x7f87d12009f9 in _start (/home/user/a.out+0x9f9)  
  
Address 0x7fffe28a4a1c is located in stack of thread T0 at offset 44 in frame  
#0 0x7f87d1200ae9 in main overflow.c:3  
  
This frame has 1 object(s):  
[32, 44) 'a' <== Memory access at offset 44 overflows this variable  
HINT: this may be a false positive if your program uses some custom stack unwind mechanism or  
swapcontext  
(longjmp and C++ exceptions *are* supported)  
SUMMARY: AddressSanitizer: stack-buffer-overflow overflow.c:6 in main  
Shadow bytes around the buggy address:  
...  
0x10007c50c930: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1  
=>0x10007c50c940: f1 f1 00[04]f2 f2 00 00 00 00 00 00 00 00 00 00
```

总结

性能分析和调试工具目标

- 性能工具和调试工具：
 - 普遍性：能够分析各种问题
 - 自动性：自动定位问题
 - 扩展性：可用于大规模并行程序
 - 易用性：无需重新编译应用程序、结果易于理解
 - 开销：性能开销小

正确使用工具

- 工具会带来额外**程序开销**
 - 重编译、反复运行、前后现象不一致等
 - 性能开销
 - PIN 30-100 倍性能下降
 - 确定可以容忍的程度
- 工具会增加**程序员开销**
 - 确定可以容忍的程度
- 选用正确的工具

分析具体场景

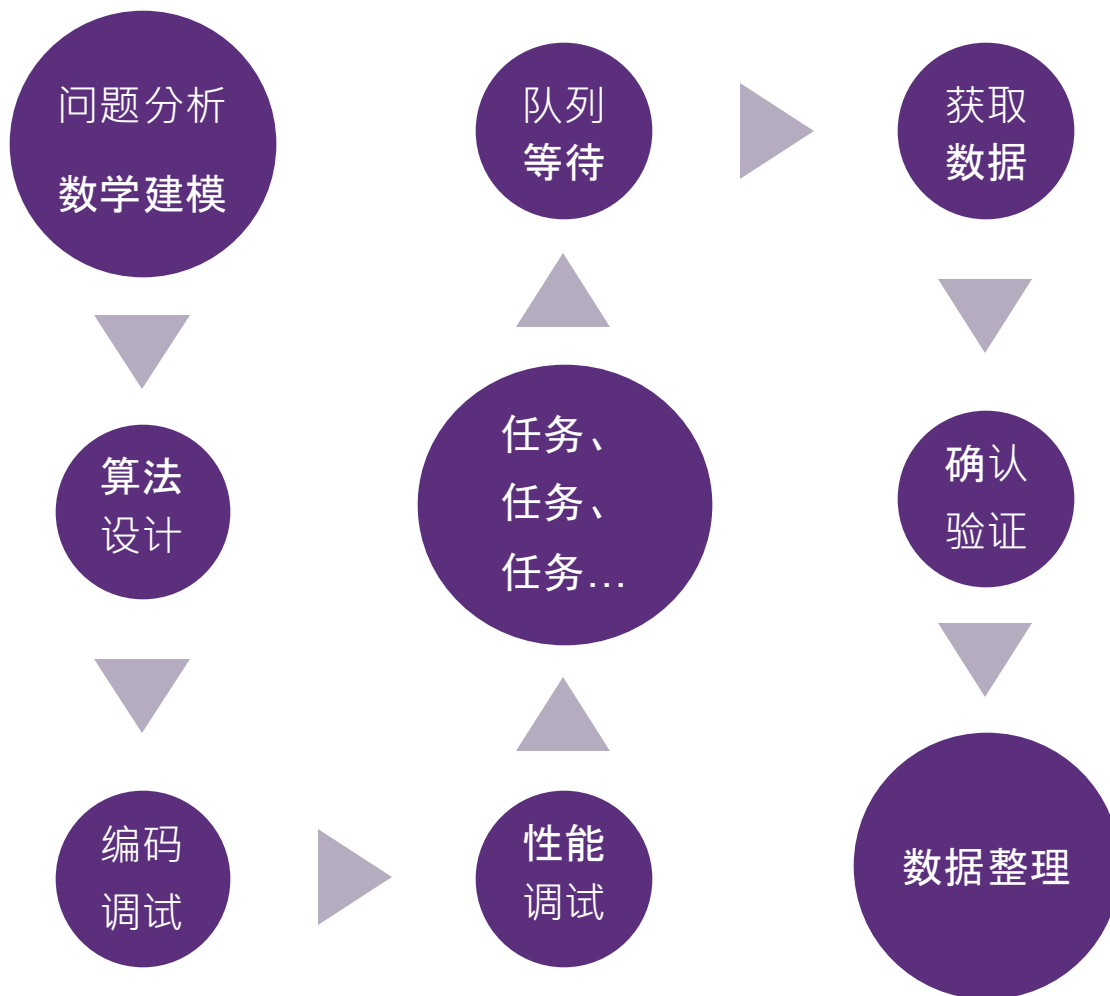
- 优化一个“慢”的代码（**未知问题**）：vTune、GProf
- 优化一个具体的性能问题（**已知问题**）：vTune、IPM、ITC/ITA
- 监控生产集群的性能（**监控问题**）：PMU

总结

- 调试和性能分析是困难的
- 熟练使用工具可以节约大量时间
- 需要做到
 1. 正确、适时地使用工具
 2. 熟悉常见的程序错误和性能问题
 3. 重视性能的理论分析
 4. 结合体系结构，确定正确的分析方向

总结：性能是一个系统工程

- 性能涉及系统每一个环节



性能工具资料

- HPCToolkit:

- <http://hpctoolkit.org/>

- PAPI:

- <http://icl.cs.utk.edu/papi/>

- Vampir:

- <https://vampir.eu/>

- Scalasca:

- <http://www.scalasca.org/>

- TotalView:

- <https://totalview.io/products/totalview>

- PIN:

- <https://software.intel.com/content/www/us/en/develop/articles/pin-a-dynamic-binary-instrumentation-tool.html>

课后练习

- 了解并使用 42 页中的性能工具
- 阅读 perf 的文档并使用
- 学习使用 vTune 性能工具
- 学习 IPM 工具的使用

小作业-9

- 使用IPM和vTune，对一个黑盒程序进行性能分析
- 使用IPM进行轻量级性能分析，检测MPI热点
- 使用vTune采集PMU数据，分析程序行为
- **提示：**亲自动手进行测试
- 详见网络学堂